Alumno: Luis Rodriguez Palacios

Modulo: Entornos de Desarrollo

Ciclo: Grado superior Desarrollo de Aplicaciones Multiplataforma

Fecha: 28/11/2025

Profesor: Cesar Brera Marquez

Titulo: Documentacion

Indice:

REFACTOR..... 4

1. Crear un proyecto Eclipse llamado “refactor”. 4
2. Crear un paquete “utilidades” y, dentro, una clase Circulo. 4
3. Crear un paquete “figuras” y, dentro, una clase Test. 5
4. Mediante las utilidades de "Código fuente": 5
5. Mover la clase Circulo al paquete figuras. 10
6. Renombrar la clase Circulo por Circunferencia. Observar si el cambio afecta a otras clases (en este caso Test). 11
7. Renombrar el atributo "rad" por "radio". ¿Cómo afecta al método get?..... 13
8. Convertir la variable local "color" del método imprimir en un atributo, inicializando su valor en el mismo método imprimir. 14
9. En imprimir, en lugar de calcular y escribir el diámetro directamente en el println, extraer a una variable local "d" e imprimir dicha variable..... 15
10. Hacer que 3.1416 sea una constante llamada PI. 16
11. Extraer el cálculo del área a un método llamado calcularArea. No recibirá parámetros y devolverá un double. 19
12. Cambiar la firma o cabecera del método esIgual, invirtiendo el orden de los parámetros y cambiando el nombre de conDecimales por considerarDecimales. ¿Cómo afecta el cambio a la clase Test, en la que se usaba este método? 23
13. Ahora usar "inline" para deshacer algunos cambios: seleccionar la variable "d" (diámetro) y hacer que su valor se use en línea, desapareciendo por tanto la variable. 26
14. Seleccionar la llamada al método calcularArea y hacer que su código se incorpore en la misma línea, desapareciendo la necesidad de usar el método..... 29
15. Seleccionar la constante PI y hacer que su valor se incorpore a las líneas en que se usa, desapareciendo por tanto la constante..... 32
16. Intentar hacer la misma operación con el atributo "color". ¿Es posible eliminar un atributo y utilizar su valor en línea? 35

DOCUMENTACION 35

1. Captura del interior del directorio para ver lo que ha generado. 35
2. Captura de cómo se ve el index.html 36
3. Captura de la documentación de la Calculadora que se vean todos los métodos. 37

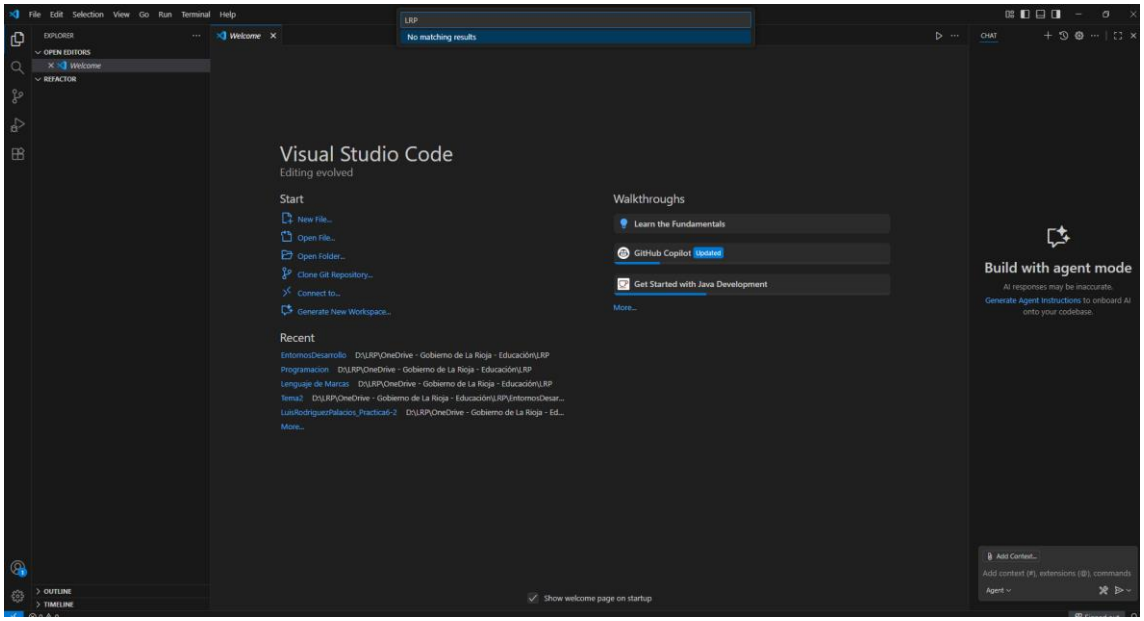
¿Cómo lo generamos para que salga en la documentación el autor y la versión?..... 37

4. Captura de la nueva documentación donde se ve el autor y la versión. 38

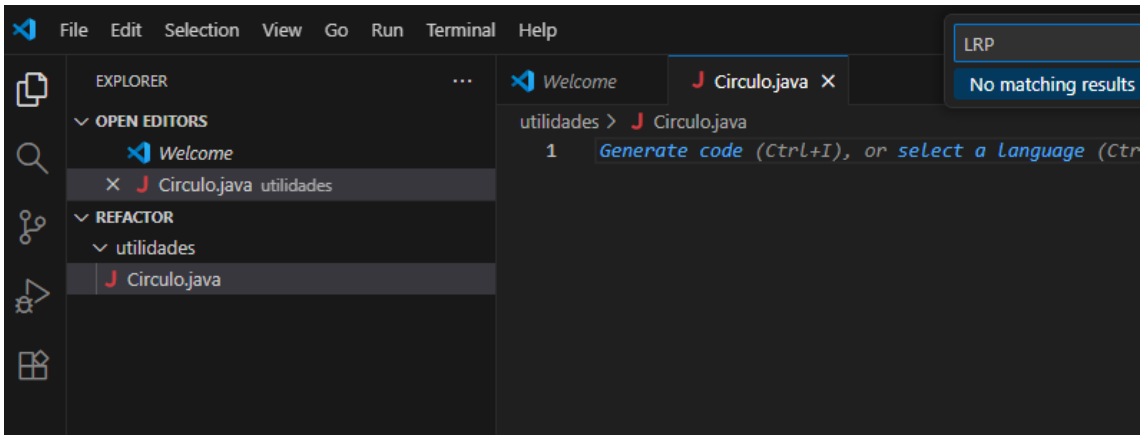
MARKDOWN 38
Cursivas y negrita 39
Encabezados 40
Enlaces 41
Imagenes 42
Citas 44
Listas 45
Párrafos 46

REFACTOR

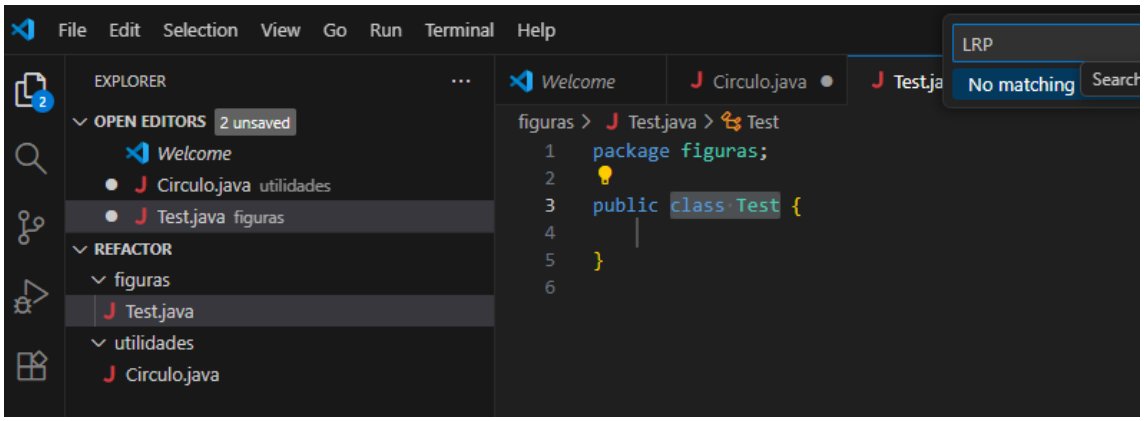
1. Crear un proyecto Eclipse llamado “refactor”.



2. Crear un paquete “utilidades” y, dentro, una clase Circulo.



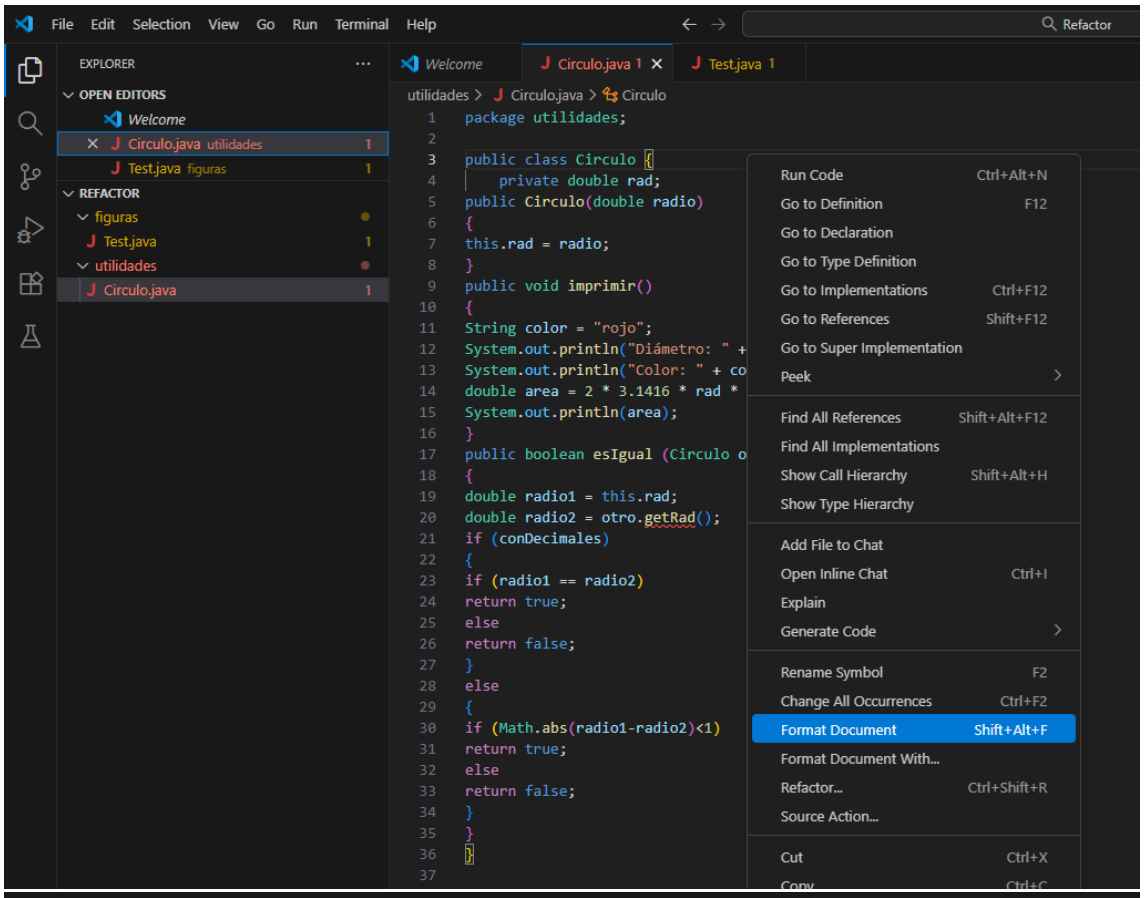
3. Crear un paquete “figuras” y, dentro, una clase Test.

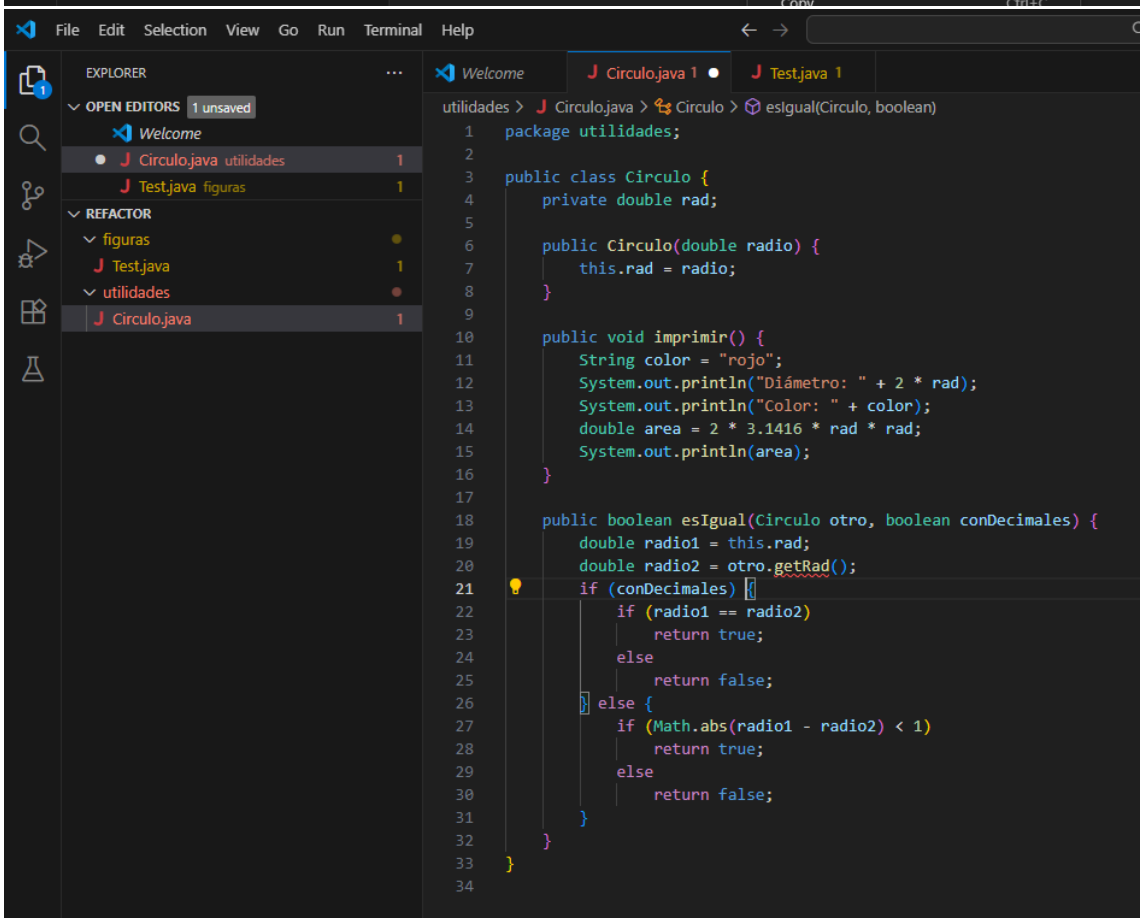


4. Mediante las utilidades de "Código fuente":
   - Dar formato al código.
   - Corregir la tabulación del código.

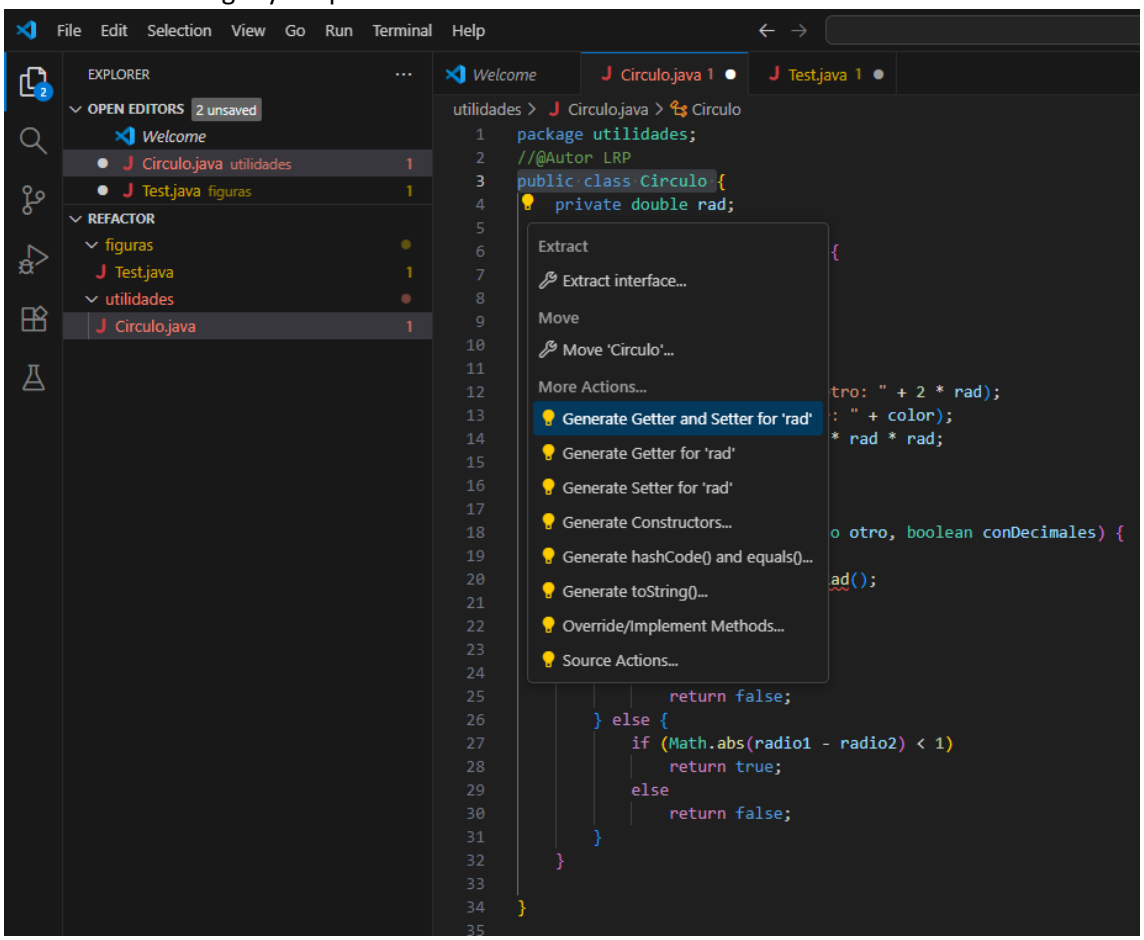
```
package utilidades;

public class Circulo {

    Extract
    Extract interface...
    Move
    Move 'Circulo'...
    More Actions...
    Generate Getter and Setter for 'rad'
    Generate Getter for 'rad'
    Generate Setter for 'rad'
    Generate Constructors...
    Generate hashCode() and equals()...
    Generate toString()...
    Override/Implement Methods...
    Source Actions...

                {
                tro: " + 2 * rad);
                : " + color);
                * rad * rad;

                o otro, boolean conDecimales) {

                ad();

                else
                    return false;
            } else {
                if (Math.abs(radio1 - radio2) < 1)
                    return true;
                else
                    return false;
            }
        }
    }
```

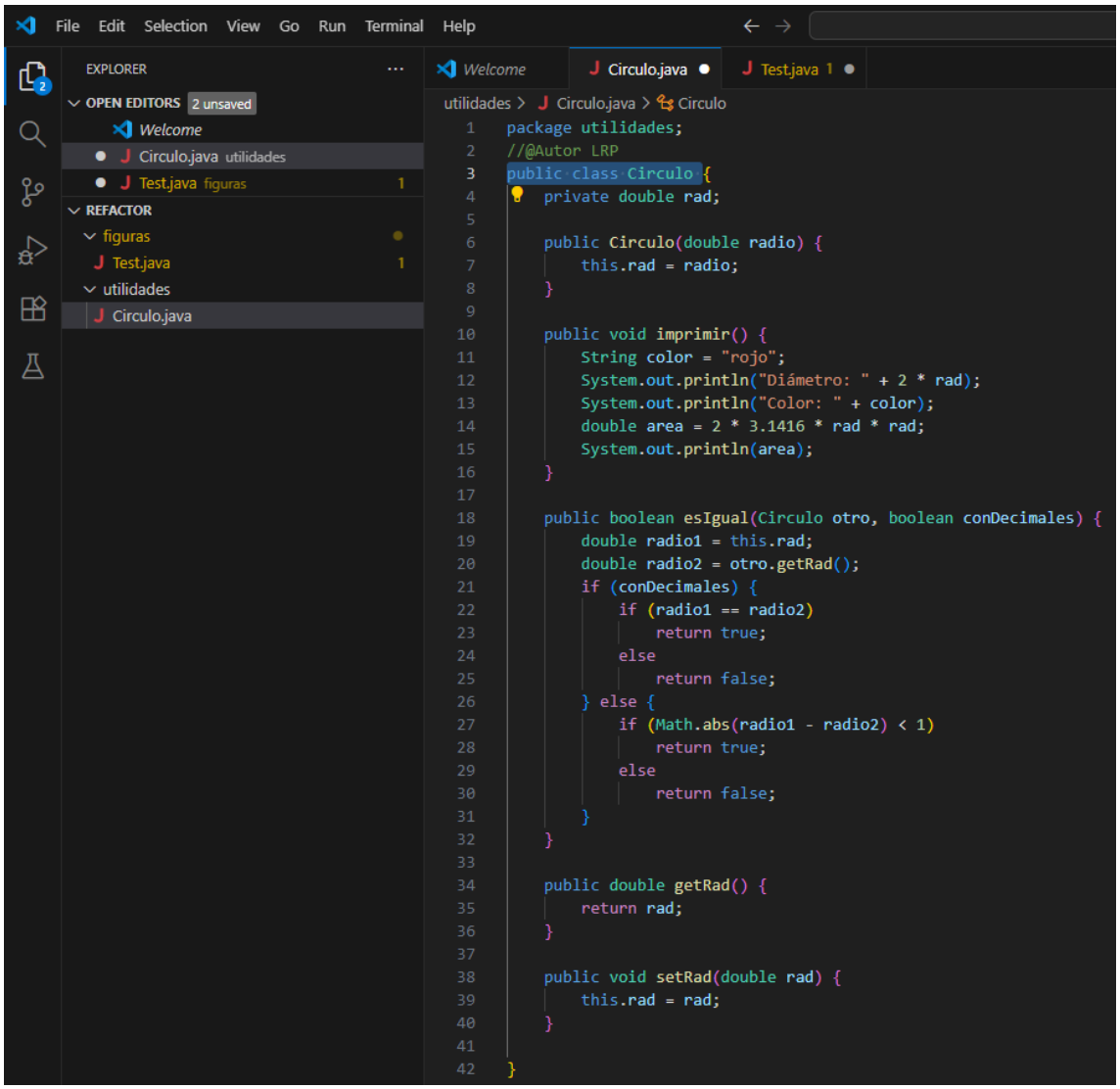
```
        }
    }
                double utilidades.Circulo.getRad()
    public double getRad() {
        return rad;
    }

    public void setRad(double rad) {
        this.rad = rad;
    }
}
```


File Edit Selection View Go Run Terminal Help
Refactor
EXPLORER
OPEN EDITORS
Welcome
Circulo.java utilidades 1
Test.java figuras 1
REFACTOR
figuras
Test.java 1
utilidades
Circulo.java 1
Welcome
Circulo.java 1
Test.java 1
utilidades > Circulo.java > Circulo
package utilidades;
public class Circulo {
private double rad;
public Circulo(double radio)
{
this.rad = radio;
}
public void imprimir()
{
String color = "rojo";
System.out.println("Diámetro: " +
System.out.println("Color: " + co
double area = 2 * 3.1416 * rad *
System.out.println(area);
}
public boolean esIgual (Circulo o
{
double radio1 = this.rad;
double radio2 = otro.getRad();
if (conDecimales)
{
if (radio1 == radio2)
return true;
else
return false;
}
else
{
if (Math.abs(radio1-radio2)<1)
return true;
else
return false;
}
}
}
Run Code Ctrl+Alt+N
Go to Definition F12
Go to Declaration
Go to Type Definition
Go to Implementations Ctrl+F12
Go to References Shift+F12
Go to Super Implementation
Peek
Find All References Shift+Alt+F12
Find All Implementations
Show Call Hierarchy Shift+Alt+H
Show Type Hierarchy
Add File to Chat
Open Inline Chat Ctrl+I
Explain
Generate Code
Rename Symbol F2
Change All Occurrences Ctrl+F2
Format Document Shift+Alt+F
Format Document With...
Refactor... Ctrl+Shift+R
Source Action...
Cut Ctrl+X

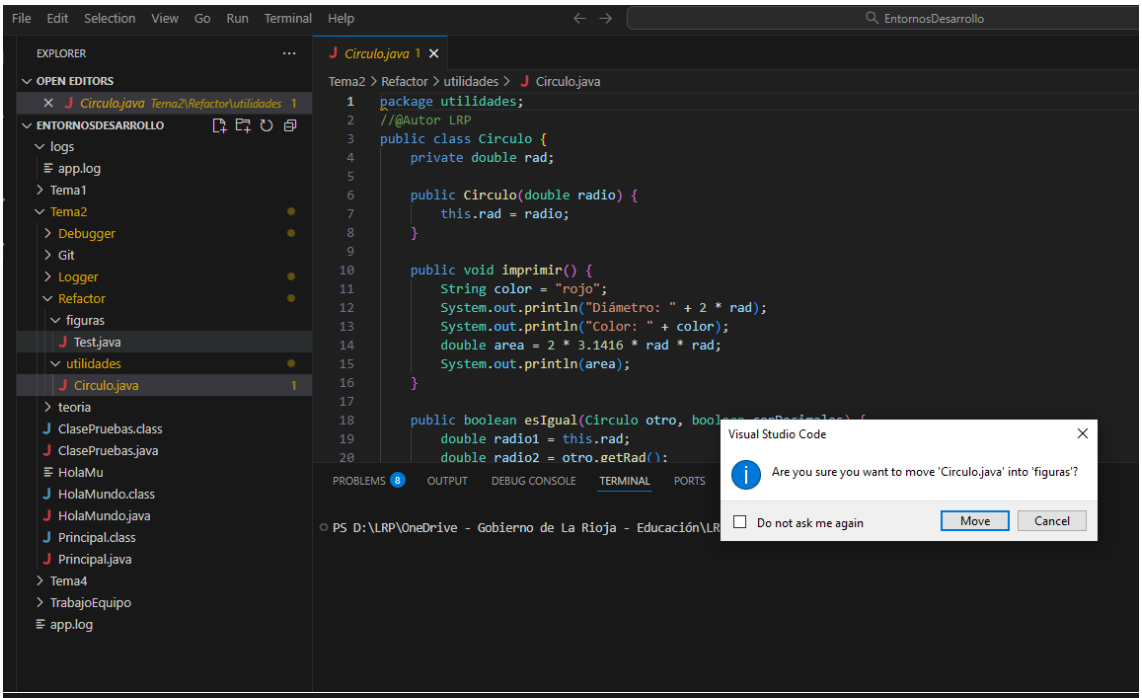

File Edit Selection View Go Run Terminal Help
EXPLORER
OPEN EDITORS 1 unsaved
Welcome
Circulo.java utilidades 1
Test.java figuras 1
REFACTOR
figuras
Test.java 1
utilidades
Circulo.java 1
Welcome
Circulo.java 1
Test.java 1
utilidades > Circulo.java > Circulo > esIgual(Circulo, boolean)
package utilidades;
public class Circulo {
private double rad;
public Circulo(double radio) {
this.rad = radio;
}
public void imprimir() {
String color = "rojo";
System.out.println("Diámetro: " + 2 * rad);
System.out.println("Color: " + color);
double area = 2 * 3.1416 * rad * rad;
System.out.println(area);
}
public boolean esIgual(Circulo otro, boolean conDecimales) {
double radio1 = this.rad;
double radio2 = otro.getRad();
if (conDecimales) {
if (radio1 == radio2)
return true;
else
return false;
} else {
if (Math.abs(radio1 - radio2) < 1)
return true;
else
return false;
}
}
}

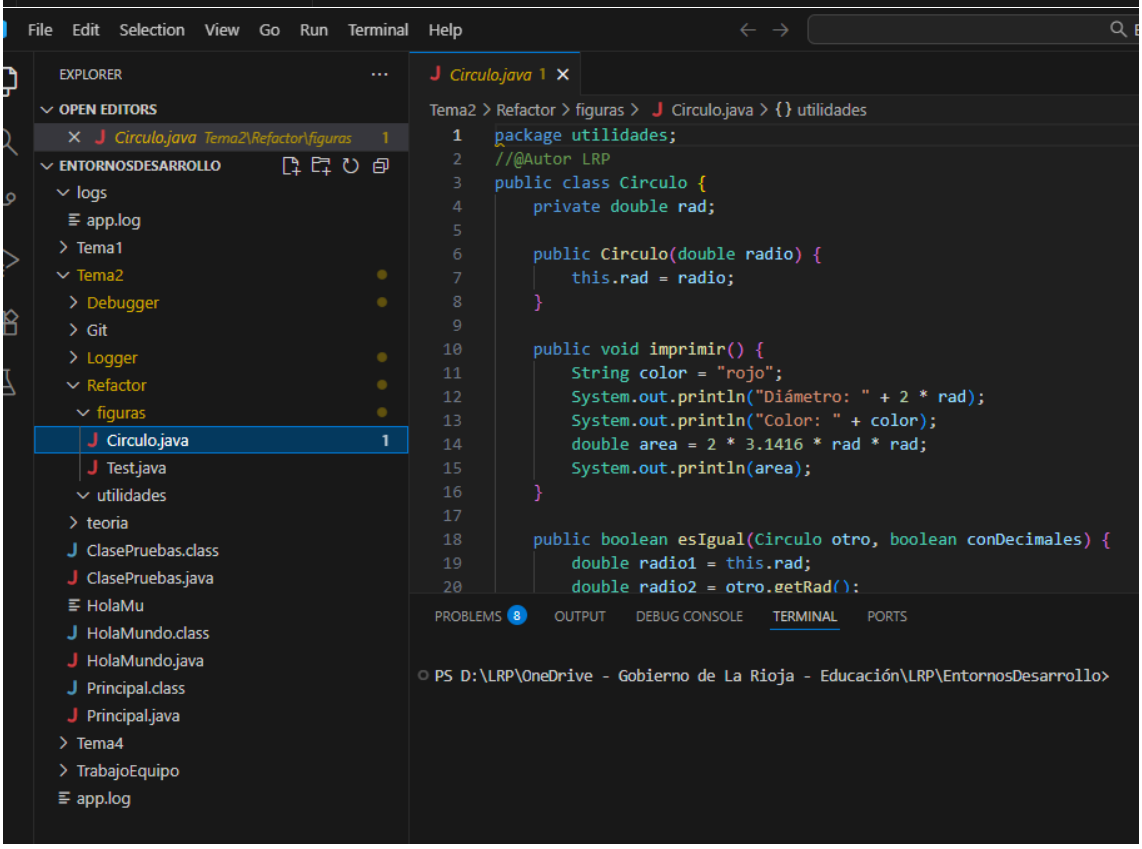
Generar métodos get y set para la clase Circulo.



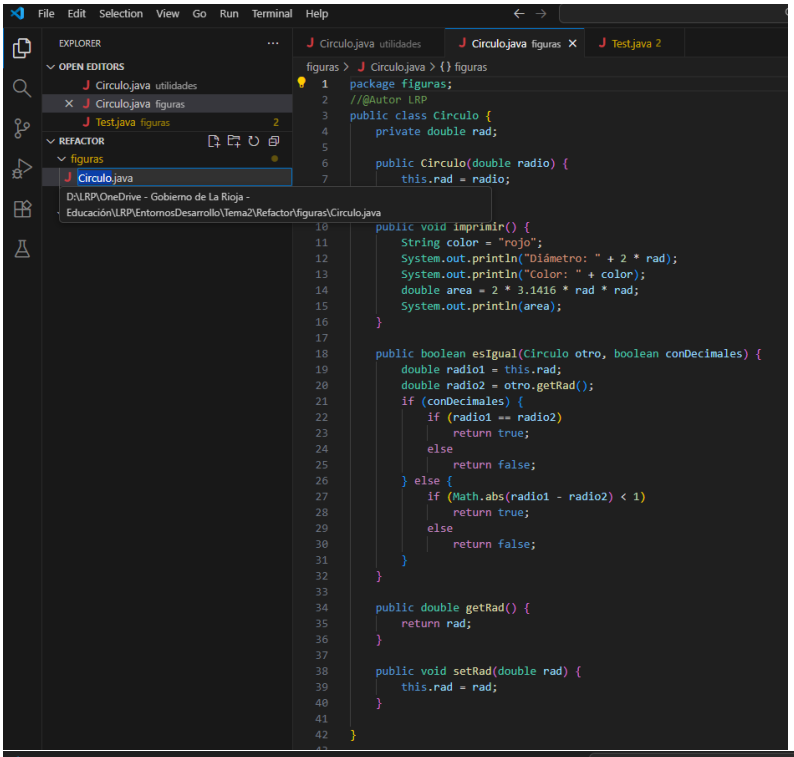

File Edit Selection View Go Run Terminal Help
EXPLORER
OPEN EDITORS 2 unsaved
Welcome
Circulo.java utilidades
Test.java figuras 1
REFACTOR
figuras
Test.java 1
utilidades
Circulo.java
Welcome
Circulo.java
Test.java 1
utilidades > Circulo.java > Circulo
package utilidades;
//@Autor LRP
public class Circulo {
private double rad;
public Circulo(double radio) {
this.rad = radio;
}
public void imprimir() {
String color = "rojo";
System.out.println("Diámetro: " + 2 * rad);
System.out.println("Color: " + color);
double area = 2 * 3.1416 * rad * rad;
System.out.println(area);
}
public boolean esIgual(Circulo otro, boolean conDecimales) {
double radio1 = this.rad;
double radio2 = otro.getRad();
if (conDecimales) {
if (radio1 == radio2)
return true;
else
return false;
} else {
if (Math.abs(radio1 - radio2) < 1)
return true;
else
return false;
}
}
public double getRad() {
return rad;
}
public void setRad(double rad) {
this.rad = rad;
}
}

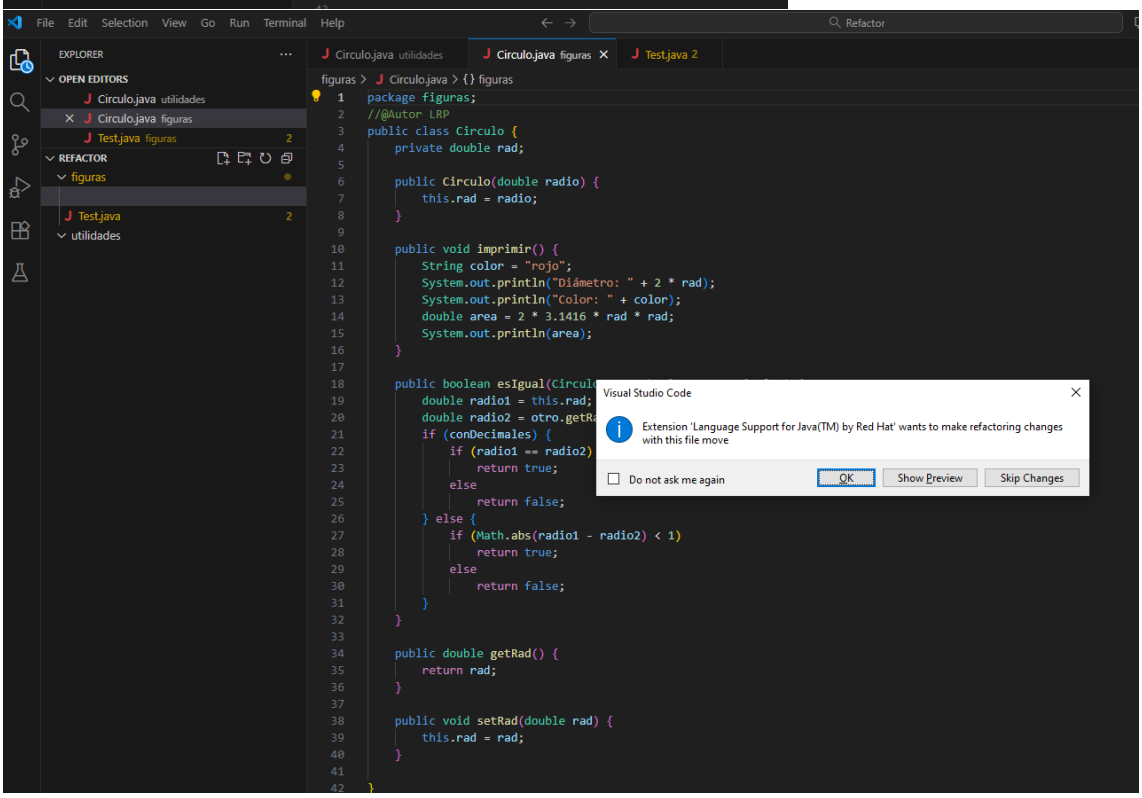
5. Mover la clase Circulo al paquete figuras.





6. Renombrar la clase Circulo por Circunferencia. Observar si el cambio afecta a otras clases (en este caso Test).





```java
public class Circunferencia {

    public Circunferencia(double radio) {
        this.rad = radio;
    }

    public void imprimir() {
        String color = "rojo";
        System.out.println("Diámetro: " + 2 * rad);
        System.out.println("Color: " + color);
        double area = 2 * 3.1416 * rad * rad;
        System.out.println(area);
    }

    public boolean esIgual(Circunferencia otro, boolean conDecimales) {
        double radio1 = this.rad;
        double radio2 = otro.getRad();
        if (conDecimales) {
            if (radio1 == radio2)
                return true;
            else
                return false;
        } else {
            if (Math.abs(radio1 - radio2) < 1)
                return true;
            else
                return false;
        }
    }

    public double getRad() {
        return rad;
    }

    public void setRad(double rad) {
        this.rad = rad;
    }

}
```

```java
package figuras;

import figuras.Circunferencia;

public class Test {
    Run | Debug
    public static void main(String[] args) {
        Circunferencia c1 = new Circunferencia(radio: 5.5);
        Circunferencia c2 = new Circunferencia(radio: 10.1);
        Circunferencia c3 = new Circunferencia(radio: 10.9);
        if (c2.esIgual(c3, conDecimales: false)) {
            System.out.println(x: "c2 y c3: iguales sin considerar decimales");
        }
        if (c2.esIgual(c3, conDecimales: true)) {
            System.out.println(x: "c2 y c3: iguales considerando decimales");
        }
    }

}
```

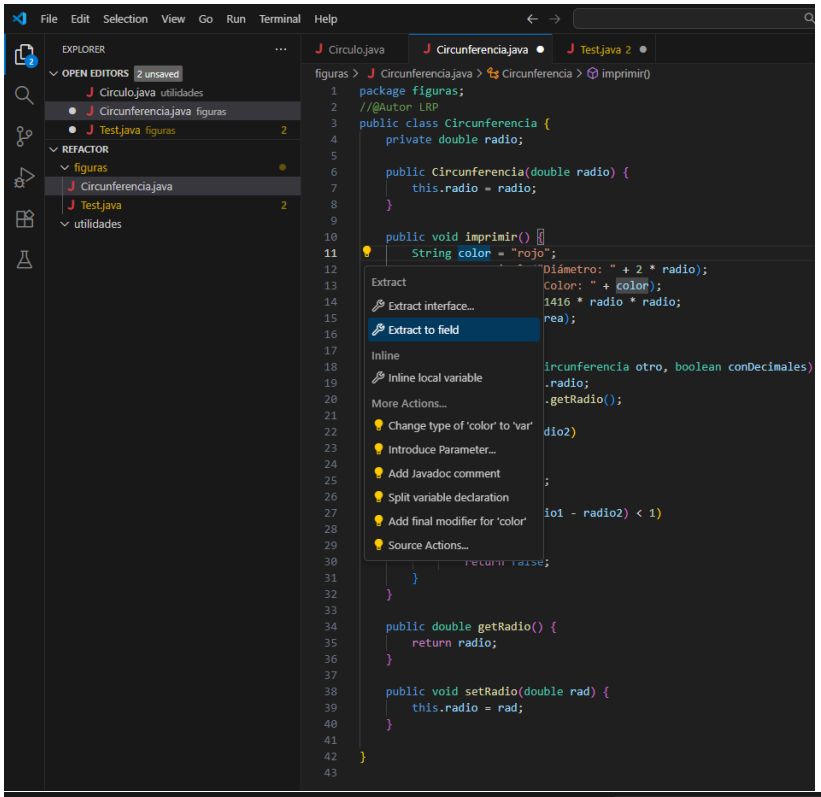
7. Renombrar el atributo "rad" por "radio". ¿Cómo afecta al método get?

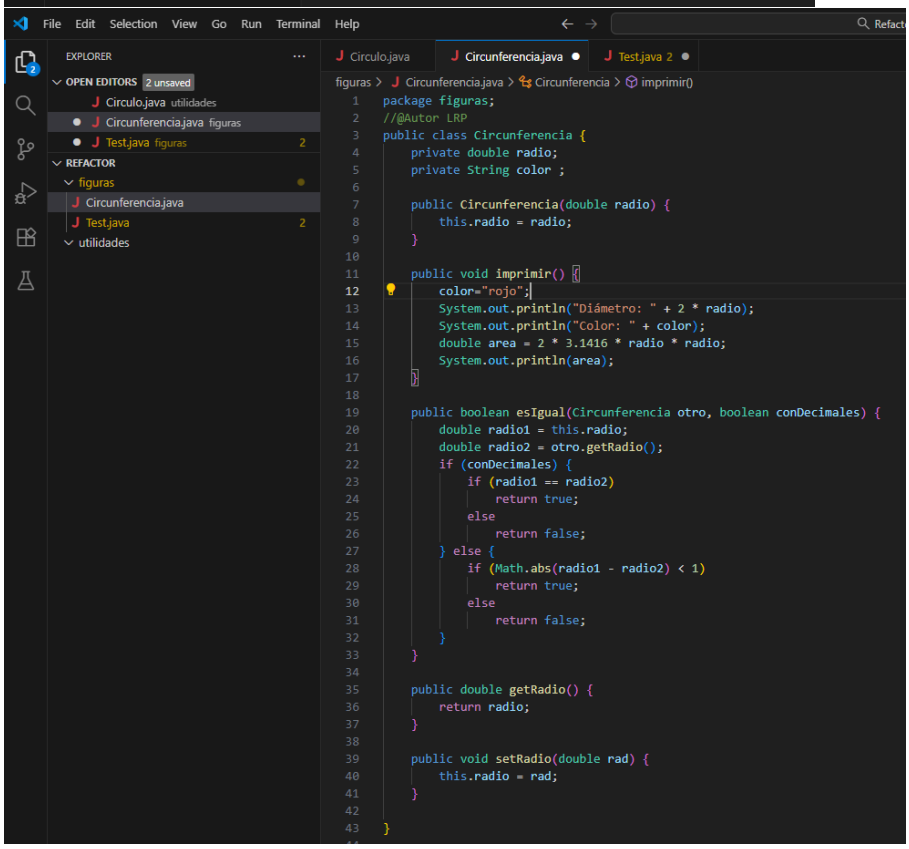
```java
package figuras;
//@Autor LRP
public class Circunferencia {
    private double rad;

    public Circunfe
        this.rad =
    }

    public void imprimir() {
        String color = "rojo";
        System.out.println("Diámetro: " + 2 * rad);
        System.out.println("Color: " + color);
        double area = 2 * 3.1416 * rad * rad;
        System.out.println(area);
    }

    public boolean esIgual(Circunferencia otro, boolean conDecimales) {
        double radio1 = this.rad;
        double radio2 = otro.getRad();
        if (conDecimales) {
            if (radio1 == radio2)
                return true;
            else
                return false;
        } else {
            if (Math.abs(radio1 - radio2) < 1)
                return true;
            else
                return false;
        }
    }

    public double getRad() {
        return rad;
    }

    public void setRad(double rad) {
        this.rad = rad;
    }

}
```

```java
package figuras;
//@Autor LRP
public class Circunferencia {
    private double radio;

    public Circunferencia(double radio) {
        this.radio = radio;
    }

    public void imprimir() {
        String color = "rojo";
        System.out.println("Diámetro: " + 2 * radio);
        System.out.println("Color: " + color);
        double area = 2 * 3.1416 * radio * radio;
        System.out.println(area);
    }

    public boolean esIgual(Circunferencia otro, boolean conDecimales) {
        double radio1 = this.radio;
        double radio2 = otro.getRadio();
        if (conDecimales) {
            if (radio1 == radio2)
                return true;
            else
                return false;
        } else {
            if (Math.abs(radio1 - radio2) < 1)
                return true;
            else
                return false;
        }
    }

    public double getRadio() {
        return radio;
    }

    public void setRadio(double rad) {
        this.radio = rad;
    }

}
```

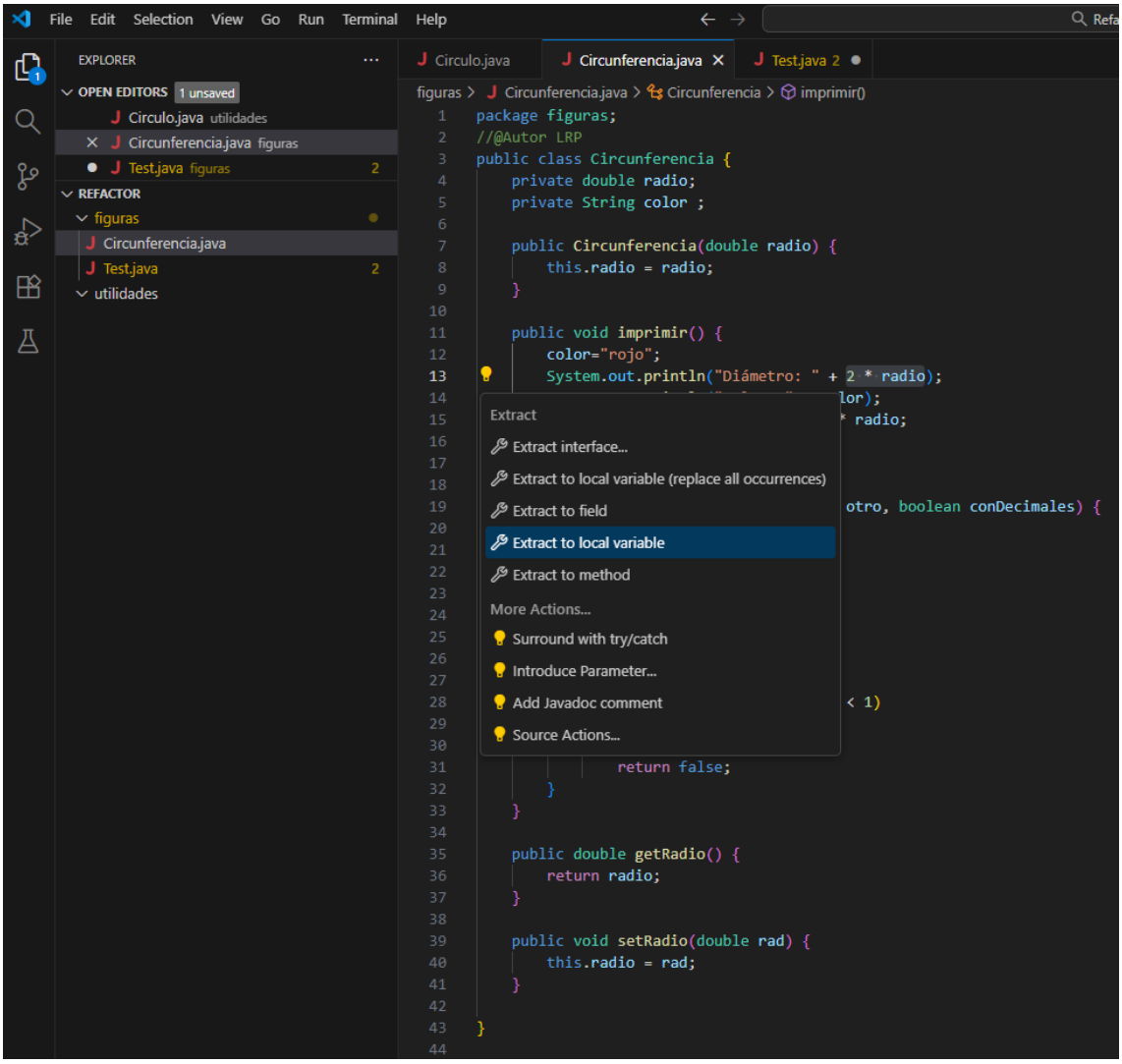
El método get, también lo cambia.

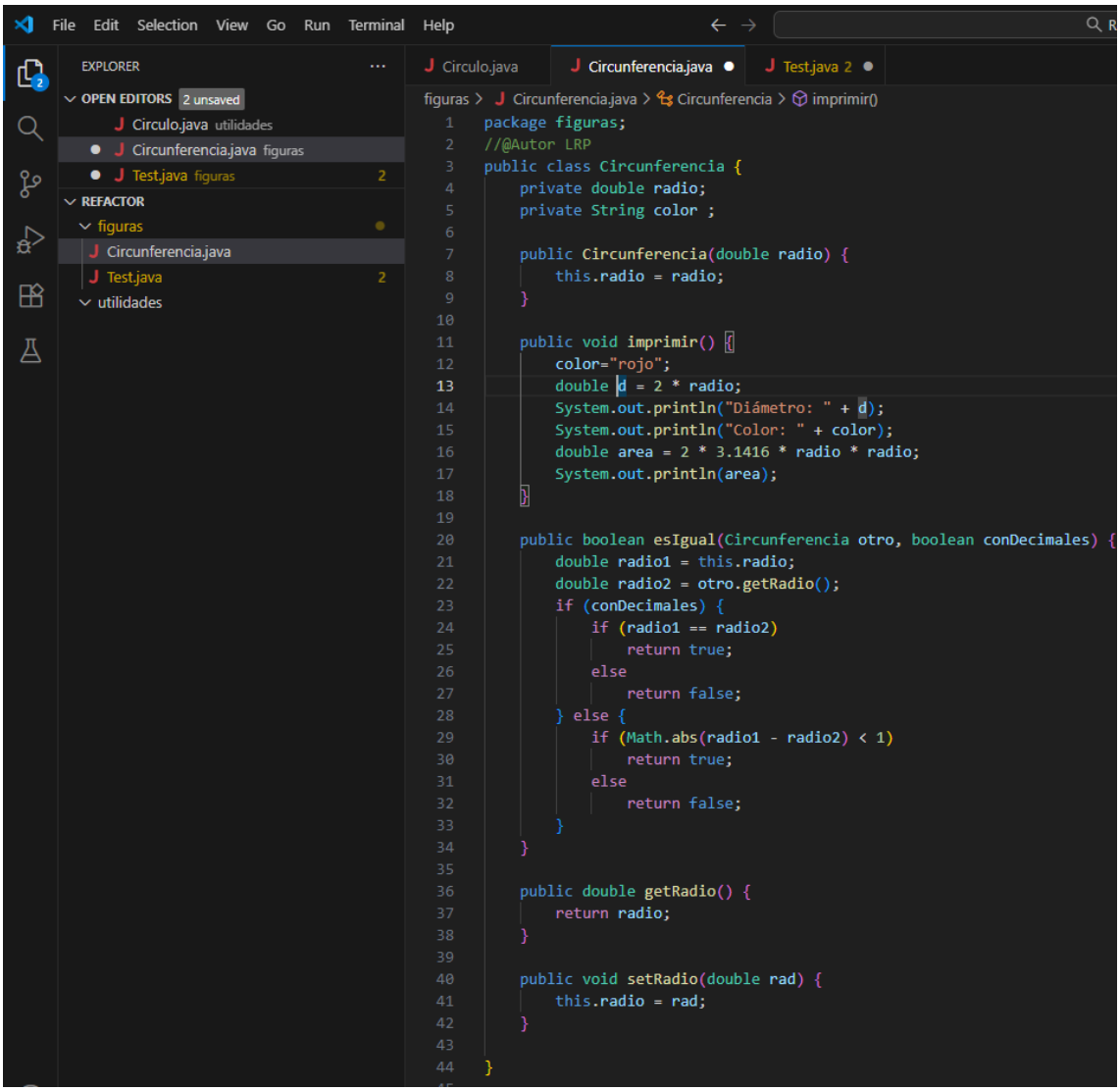
8. Convertir la variable local "color" del método imprimir en un atributo, inicializando su valor en el mismo método imprimir.





9. En imprimir, en lugar de calcular y escribir el diámetro directamente en el println, extraer a una variable local "d" e imprimir dicha variable.





10. Hacer que 3.1416 sea una constante llamada PI.

```java
package figuras;
//@Autor LRP
public class Circunferencia {
    private double radio;
    private String color ;

    public Circunferencia(double radio) {
        this.radio = radio;
    }

    public void imprimir() {
        color="rojo";
        double d = 2 * radio;
        System.out.println("Diámetro: " + d);
        System.out.println("Color: " + color);
        double area = 2 * 3.1416 * radio * radio;
```

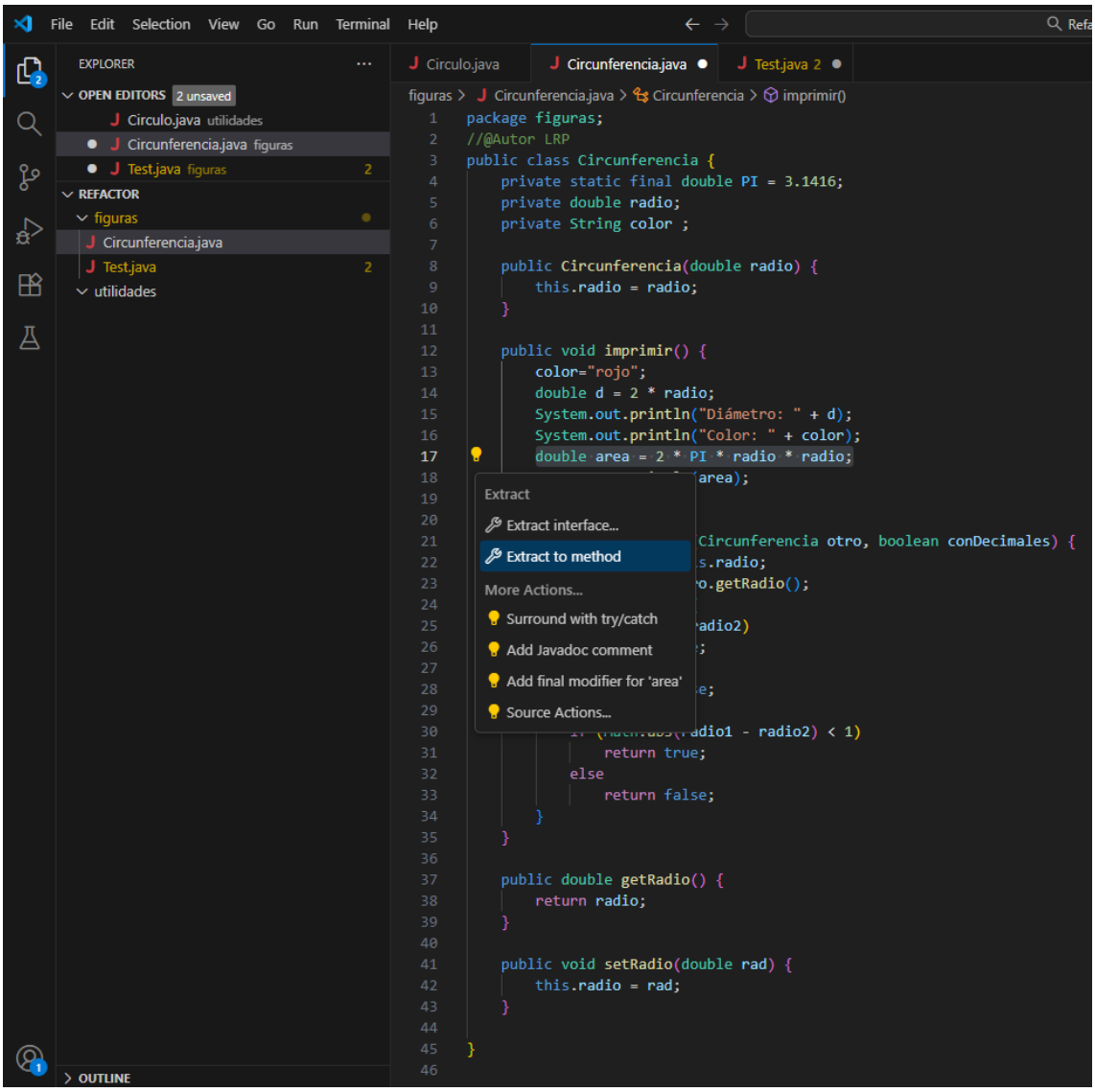
Extract
- Extract interface...
- Extract to local variable (replace all occurrences)
- Extract to field
- Extract to local variable
- Extract to constant
- Extract to method

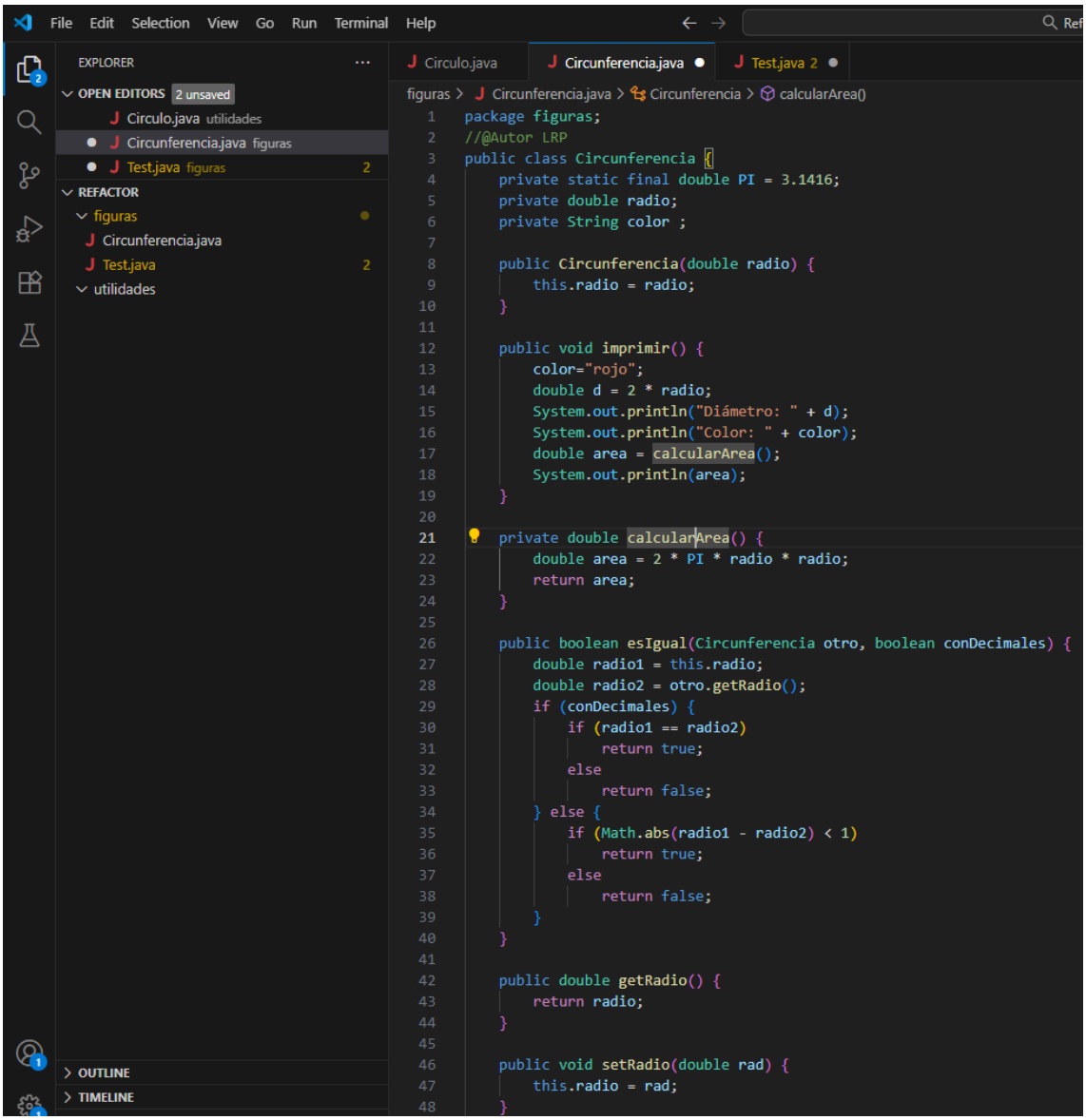
More Actions...
- Surround with try/catch
- Introduce Parameter...
- Add Javadoc comment
- Add final modifier for 'area'
- Source Actions...

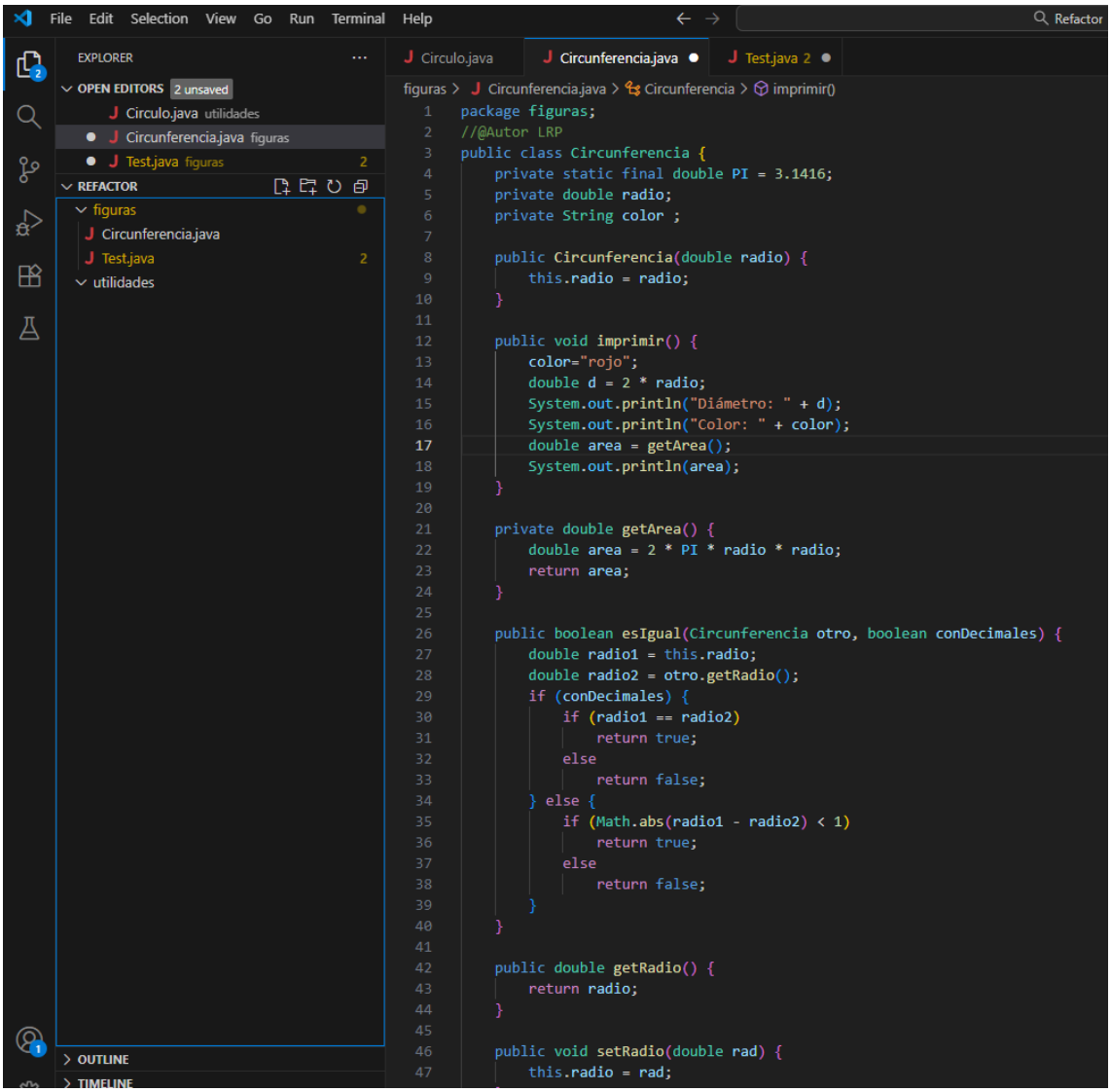
```java
        return radio;
    }

    public void setRadio(double rad) {
        this.radio = rad;
    }

}
```

```java
package figuras;
//@Autor LRP
public class Circunferencia {
    private static final double PI = 3.1416;
    private double radio;
    private String color ;

    public Circunferencia(double radio) {
        this.radio = radio;
    }

    public void imprimir() {
        color="rojo";
        double d = 2 * radio;
        System.out.println("Diámetro: " + d);
        System.out.println("Color: " + color);
        double area = 2 * PI * radio * radio;
        System.out.println(area);
    }

    public boolean esIgual(Circunferencia otro, boolean conDec
        double radio1 = this.radio;
        double radio2 = otro.getRadio();
        if (conDecimales)
            if (radio1 == radio2)
                return true;
            else
                return false;
        } else {
            if (Math.abs(radio1 - radio2) < 1)
                return true;
            else
                return false;
        }
    }

    public double getRadio() {
        return radio;
    }

    public void setRadio(double rad) {
        this.radio = rad;
    }

}
```

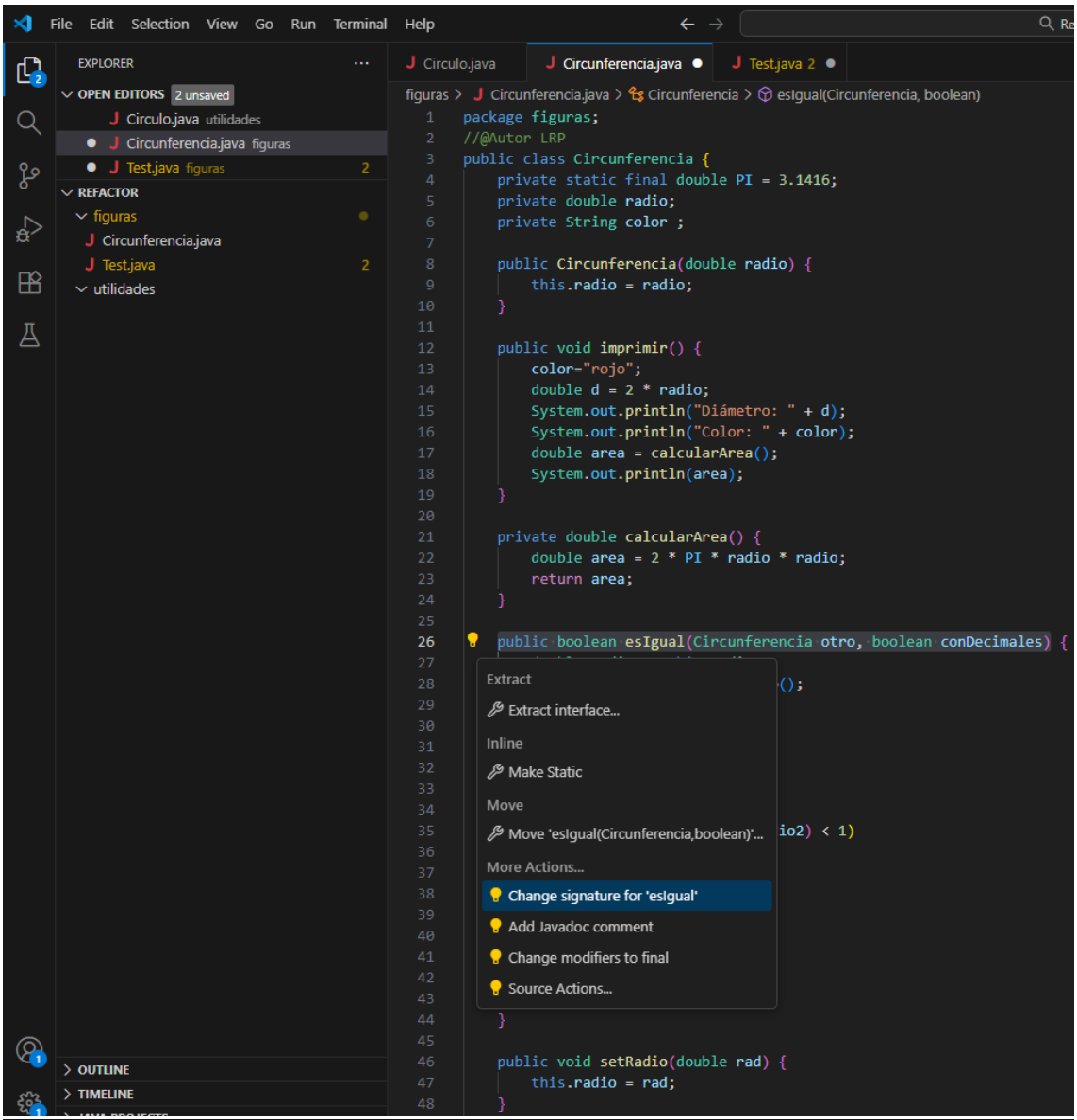
11. Extraer el cálculo del área a un método llamado calcularArea. No recibirá parámetros y devolverá un double.

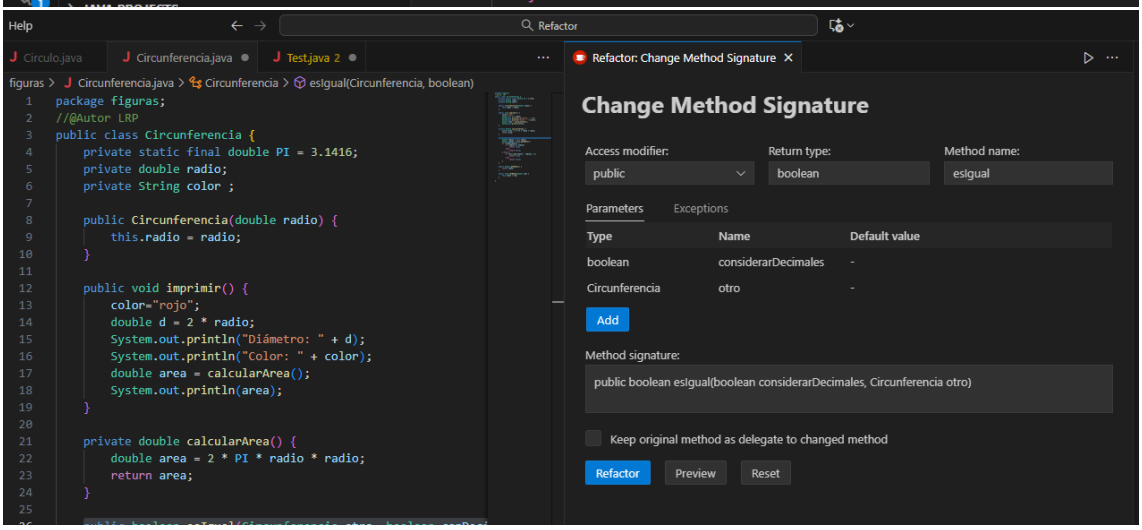

File Edit Selection View Go Run Terminal Help
EXPLORER
OPEN EDITORS 2 unsaved
Circulo.java utilidades
Circunferencia.java figuras
Test.java figuras 2
REFACTOR
figuras
Circunferencia.java
Test.java 2
utilidades
OUTLINE
Circulo.java
Circunferencia.java
Test.java 2
figuras > Circunferencia.java > Circunferencia > imprimir()
package figuras;
//@Autor LRP
public class Circunferencia {
private static final double PI = 3.1416;
private double radio;
private String color ;
public Circunferencia(double radio) {
this.radio = radio;
}
public void imprimir() {
color="rojo";
double d = 2 * radio;
System.out.println("Diámetro: " + d);
System.out.println("Color: " + color);
double area = 2 * PI * radio * radio;
Extract
Extract interface...
Extract to method
More Actions...
Surround with try/catch
Add Javadoc comment
Add final modifier for 'area'
Source Actions...
return true;
else
return false;
}
}
public double getRadio() {
return radio;
}
public void setRadio(double rad) {
this.radio = rad;
}
}


File Edit Selection View Go Run Terminal Help
EXPLORER
OPEN EDITORS 2 unsaved
Circulo.java utilidades
Circunferencia.java figuras
Test.java figuras 2
REFACTOR
figuras
Circunferencia.java
Test.java 2
utilidades
OUTLINE
TIMELINE
Circulo.java
Circunferencia.java
Test.java 2
figuras > Circunferencia.java > Circunferencia > calcularArea()
package figuras;
//@Autor LRP
public class Circunferencia {
private static final double PI = 3.1416;
private double radio;
private String color ;
public Circunferencia(double radio) {
this.radio = radio;
}
public void imprimir() {
color="rojo";
double d = 2 * radio;
System.out.println("Diámetro: " + d);
System.out.println("Color: " + color);
double area = calcularArea();
System.out.println(area);
}
private double calcularArea() {
double area = 2 * PI * radio * radio;
return area;
}
public boolean esIgual(Circunferencia otro, boolean conDecimales) {
double radio1 = this.radio;
double radio2 = otro.getRadio();
if (conDecimales) {
if (radio1 == radio2)
return true;
else
return false;
} else {
if (Math.abs(radio1 - radio2) < 1)
return true;
else
return false;
}
}
public double getRadio() {
return radio;
}
public void setRadio(double rad) {
this.radio = rad;
}


File Edit Selection View Go Run Terminal Help
Refactor
EXPLORER
OPEN EDITORS 2 unsaved
Circulo.java utilidades
Circunferencia.java figuras
Test.java figuras 2
REFACTOR
figuras
Circunferencia.java
Test.java 2
utilidades
OUTLINE
TIMELINE
Circulo.java
Circunferencia.java
Test.java 2
figuras > Circunferencia.java > Circunferencia > imprimir()
1 package figuras;
2 //@Autor LRP
3 public class Circunferencia {
4 private static final double PI = 3.1416;
5 private double radio;
6 private String color ;
7
8 public Circunferencia(double radio) {
9 this.radio = radio;
10 }
11
12 public void imprimir() {
13 color="rojo";
14 double d = 2 * radio;
15 System.out.println("Diámetro: " + d);
16 System.out.println("Color: " + color);
17 double area = getArea();
18 System.out.println(area);
19 }
20
21 private double getArea() {
22 double area = 2 * PI * radio * radio;
23 return area;
24 }
25
26 public boolean esIgual(Circunferencia otro, boolean conDecimales) {
27 double radio1 = this.radio;
28 double radio2 = otro.getRadio();
29 if (conDecimales) {
30 if (radio1 == radio2)
31 return true;
32 else
33 return false;
34 } else {
35 if (Math.abs(radio1 - radio2) < 1)
36 return true;
37 else
38 return false;
39 }
40 }
41
42 public double getRadio() {
43 return radio;
44 }
45
46 public void setRadio(double rad) {
47 this.radio = rad;

12. Cambiar la firma o cabecera del método esIgual, invirtiendo el orden de los parámetros y cambiando el nombre de conDecimales por considerarDecimales. ¿Cómo afecta el cambio a la clase Test, en la que se usaba este método?

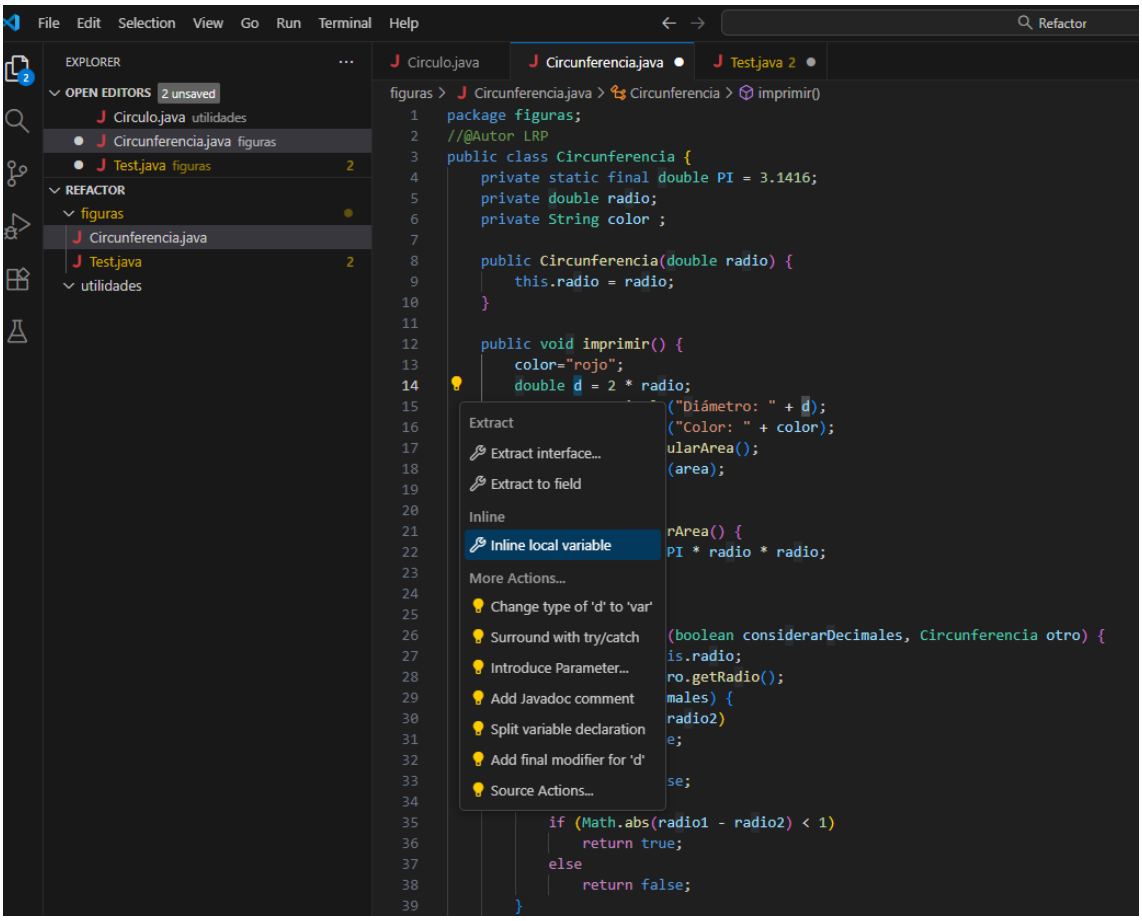

File Edit Selection View Go Run Terminal Help
EXPLORER
OPEN EDITORS 2 unsaved
Circulo.java utilidades
Circunferencia.java figuras
Test.java figuras 2
REFACTOR
figuras
Circunferencia.java
Test.java 2
utilidades
OUTLINE
TIMELINE
Circulo.java
Circunferencia.java
Test.java 2
figuras > Circunferencia.java > Circunferencia > eslgual(Circunferencia, boolean)
package figuras;
//@Autor LRP
public class Circunferencia {
private static final double PI = 3.1416;
private double radio;
private String color ;
public Circunferencia(double radio) {
this.radio = radio;
}
public void imprimir() {
color="rojo";
double d = 2 * radio;
System.out.println("Diámetro: " + d);
System.out.println("Color: " + color);
double area = calcularArea();
System.out.println(area);
}
private double calcularArea() {
double area = 2 * PI * radio * radio;
return area;
}
public boolean esIgual(Circunferencia otro, boolean conDecimales) {
Extract
Extract interface...
Inline
Make Static
Move
Move 'eslgual(Circunferencia,boolean)'...
More Actions...
Change signature for 'eslgual'
Add Javadoc comment
Change modifiers to final
Source Actions...
}
public void setRadio(double rad) {
this.radio = rad;
}

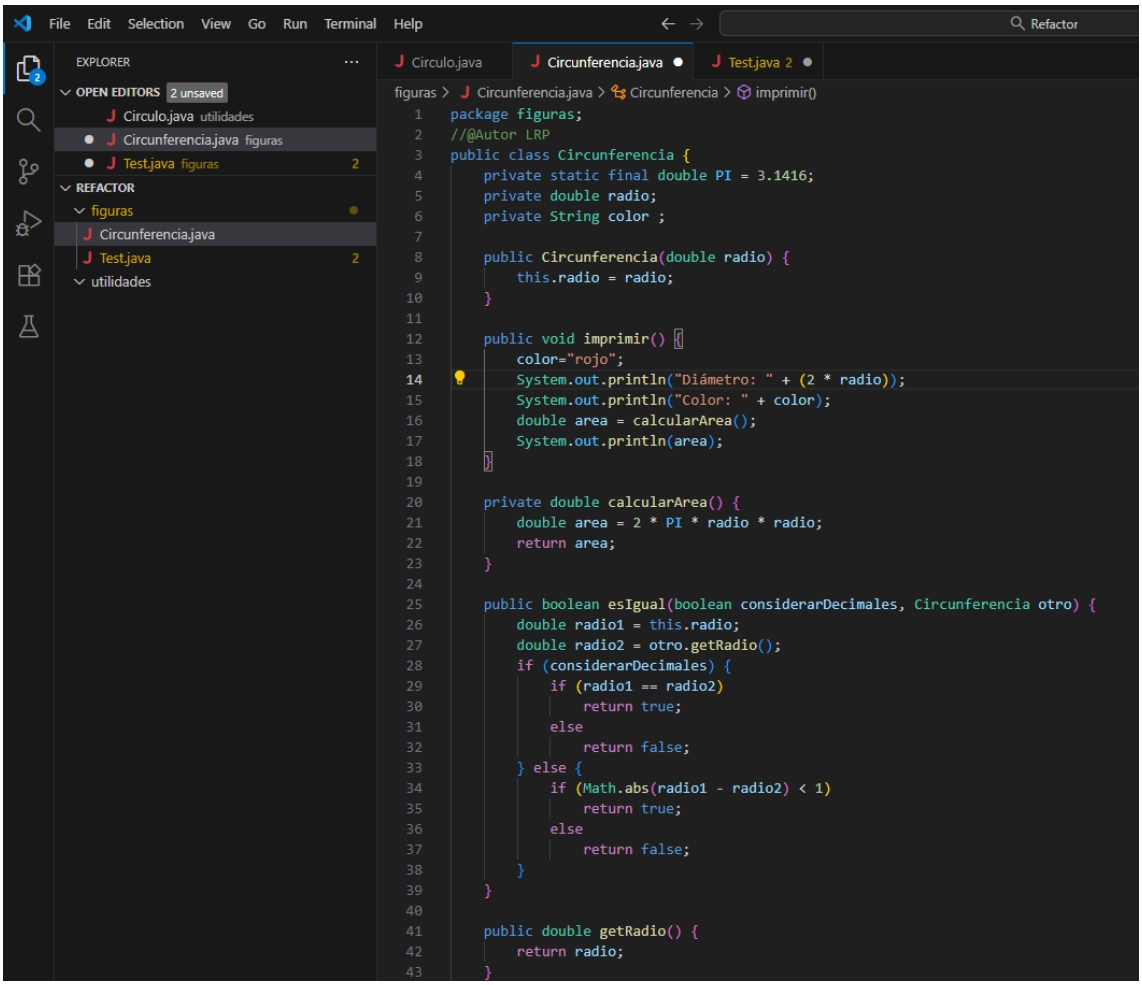

Help
Refactor
Circulo.java
Circunferencia.java
Test.java 2
Refactor: Change Method Signature
figuras > Circunferencia.java > Circunferencia > eslgual(Circunferencia, boolean)
package figuras;
//@Autor LRP
public class Circunferencia {
private static final double PI = 3.1416;
private double radio;
private String color ;
public Circunferencia(double radio) {
this.radio = radio;
}
public void imprimir() {
color="rojo";
double d = 2 * radio;
System.out.println("Diámetro: " + d);
System.out.println("Color: " + color);
double area = calcularArea();
System.out.println(area);
}
private double calcularArea() {
double area = 2 * PI * radio * radio;
return area;
}
Change Method Signature
Access modifier:
public
Return type:
boolean
Method name:
eslgual
Parameters
Exceptions
Type
Name
Default value
boolean
considerarDecimales
-
Circunferencia
otro
-
Add
Method signature:
public boolean eslgual(boolean considerarDecimales, Circunferencia otro)
Keep original method as delegate to changed method
Refactor
Preview
Reset

```java
package figuras;
//@Autor LRP
public class Circunferencia {
    private static final double PI = 3.1416;
    private double radio;
    private String color ;

    public Circunferencia(double radio) {
        this.radio = radio;
    }

    public void imprimir() {
        color="rojo";
        double d = 2 * radio;
        System.out.println("Diámetro: " + d);
        System.out.println("Color: " + color);
        double area = calcularArea();
        System.out.println(area);
    }

    private double calcularArea() {
        double area = 2 * PI * radio * radio;
        return area;
    }

    public boolean esIgual(boolean considerarDecimales, Circunferencia otro) {
        double radio1 = this.radio;
        double radio2 = otro.getRadio();
        if (considerarDecimales) {
            if (radio1 == radio2)
                return true;
```

```java
package figuras;
//@Author LRP

import figuras.Circunferencia;

public class Test {
    Run | Debug
    public static void main(String[] args) {
        Circunferencia c1 = new Circunferencia(radio: 5.5);
        Circunferencia c2 = new Circunferencia(radio: 10.1);
        Circunferencia c3 = new Circunferencia(radio: 10.9);
        if (c2.esIgual(considerarDecimales: false, c3)) {
            System.out.println(x: "c2 y c3: iguales sin considerar decimales");
        }
        if (c2.esIgual(considerarDecimales: true, c3)) {
            System.out.println(x: "c2 y c3: iguales considerando decimales");
        }
    }

}
```

13. Ahora usar "inline" para deshacer algunos cambios: seleccionar la variable "d" (diámetro) y hacer que su valor se use en línea, desapareciendo por tanto la variable.


File Edit Selection View Go Run Terminal Help
Refactor
EXPLORER
OPEN EDITORS 2 unsaved
Circulo.java utilidades
Circunferencia.java figuras
Test.java figuras 2
REFACTOR
figuras
Circunferencia.java
Test.java 2
utilidades
Circulo.java
Circunferencia.java
Test.java 2
figuras > Circunferencia.java > Circunferencia > imprimir()
package figuras;
//@Autor LRP
public class Circunferencia {
private static final double PI = 3.1416;
private double radio;
private String color ;
public Circunferencia(double radio) {
this.radio = radio;
}
public void imprimir() {
color="rojo";
double d = 2 * radio;
("Diámetro: " + d);
("Color: " + color);
ularArea();
(area);
rArea() {
PI * radio * radio;
(boolean considerarDecimales, Circunferencia otro) {
is.radio;
ro.getRadio();
males) {
radio2)
e;
se;
if (Math.abs(radio1 - radio2) < 1)
return true;
else
return false;
}
Extract
Extract interface...
Extract to field
Inline
Inline local variable
More Actions...
Change type of 'd' to 'var'
Surround with try/catch
Introduce Parameter...
Add Javadoc comment
Split variable declaration
Add final modifier for 'd'
Source Actions...


File Edit Selection View Go Run Terminal Help
Refactor
EXPLORER
OPEN EDITORS 2 unsaved
Circulo.java utilidades
Circunferencia.java figuras
Test.java figuras 2
REFACTOR
figuras
Circunferencia.java
Test.java 2
utilidades
Circulo.java
Circunferencia.java
Test.java 2
figuras > Circunferencia.java > Circunferencia > imprimir()
package figuras;
//@Autor LRP
public class Circunferencia {
private static final double PI = 3.1416;
private double radio;
private String color ;
public Circunferencia(double radio) {
this.radio = radio;
}
public void imprimir() {
color="rojo";
System.out.println("Diámetro: " + (2 * radio));
System.out.println("Color: " + color);
double area = calcularArea();
System.out.println(area);
}
private double calcularArea() {
double area = 2 * PI * radio * radio;
return area;
}
public boolean esIgual(boolean considerarDecimales, Circunferencia otro) {
double radio1 = this.radio;
double radio2 = otro.getRadio();
if (considerarDecimales) {
if (radio1 == radio2)
return true;
else
return false;
} else {
if (Math.abs(radio1 - radio2) < 1)
return true;
else
return false;
}
}
public double getRadio() {
return radio;
}

14. Seleccionar la llamada al método calcularArea y hacer que su código se incorpore en la misma línea, desapareciendo la necesidad de usar el método.

File Edit Selection View Go Run Terminal Help

Refactor

EXPLORER

OPEN EDITORS 2 unsaved

Circulo.java utilidades

Circunferencia.java figuras

Test.java figuras 2

REFACTOR

figuras

Circunferencia.java

Test.java 2

utilidades

OUTLINE

TIMELINE

JAVA PROJECTS

Circulo.java | Circunferencia.java | Test.java 2

figuras > Circunferencia.java > Circunferencia > calcularArea()

```
1  package figuras;
2  //@Autor LRP
3  public class Circunferencia {
4      private static final double PI = 3.1416;
5      private double radio;
6      private String color ;
7
8      public Circunferencia(double radio) {
9          this.radio = radio;
10     }
11
12     public void imprimir() {
13         color="rojo";
14         System.out.println("Diámetro: " + (2 * radio));
15         System.out.println("Color: " + color);
16         double area = calcularArea();
17         System.out.println(area);
18     }
19
20     private double calcularArea() {
21                                 adio * radio;
22
23
24
25                                 an considerarDecimales, Circunferencia otro) {
26                                 io;
27                                 Radio();
28                                 {
29                                 )
30
31
32
33
34                                 - radio2) < 1)
35
36
37
38         }
39     }
40
41     public double getRadio() {
42         return radio;
43     }
44
45     public void setRadio(double rad) {
46         this.radio = rad;
47     }
48
49 }
```

Extract

Extract interface...

Inline

Make Static

Inline Method

Move

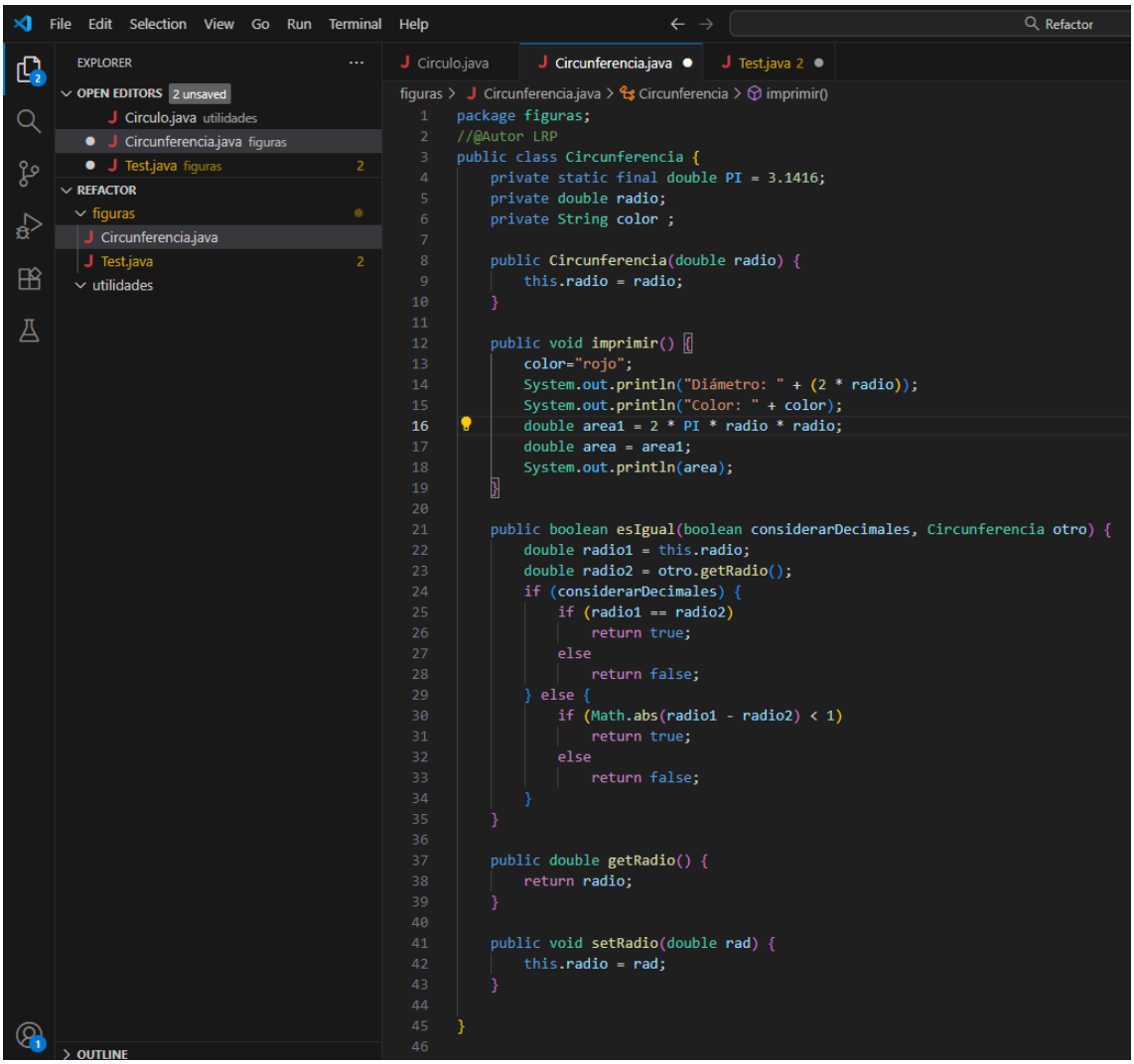
Move 'calcularArea()'...

More Actions...

Change signature for 'calcularArea'

Add Javadoc comment

Source Actions...


File Edit Selection View Go Run Terminal Help
Refactor
EXPLORER
OPEN EDITORS 2 unsaved
Circulo.java utilidades
Circunferencia.java figuras
Test.java figuras 2
REFACTOR
figuras
Circunferencia.java
Test.java 2
utilidades
OUTLINE
Circulo.java
Circunferencia.java
Test.java 2
figuras > Circunferencia.java > Circunferencia > imprimir()
package figuras;
//@Autor LRP
public class Circunferencia {
private static final double PI = 3.1416;
private double radio;
private String color ;
public Circunferencia(double radio) {
this.radio = radio;
}
public void imprimir() {
color="rojo";
System.out.println("Diámetro: " + (2 * radio));
System.out.println("Color: " + color);
double area1 = 2 * PI * radio * radio;
double area = area1;
System.out.println(area);
}
public boolean esIgual(boolean considerarDecimales, Circunferencia otro) {
double radio1 = this.radio;
double radio2 = otro.getRadio();
if (considerarDecimales) {
if (radio1 == radio2)
return true;
else
return false;
} else {
if (Math.abs(radio1 - radio2) < 1)
return true;
else
return false;
}
}
public double getRadio() {
return radio;
}
public void setRadio(double rad) {
this.radio = rad;
}
}

15. Seleccionar la constante PI y hacer que su valor se incorpore a las líneas en que se usa, desapareciendo por tanto la constante.

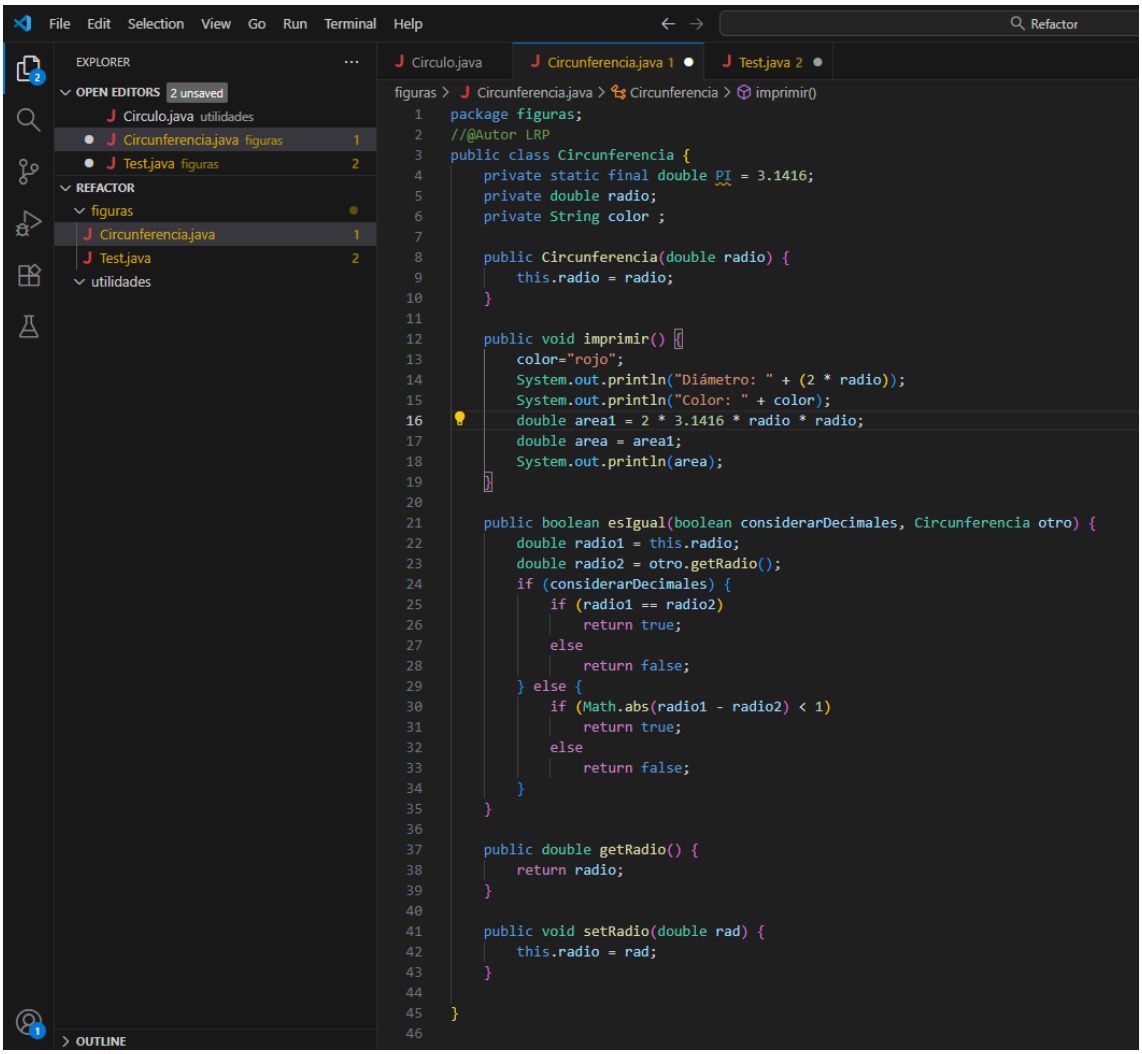
```java
package figuras;
//@Autor LRP
public class Circunferencia {
    private static final double PI = 3.1416;
    private double radio;
    private String color ;

    public Circunferencia(double radio) {
        this.radio = radio;
    }

    public void imprimir() {
        color="rojo";
        System.out.println("Diámetro: " + (2 * radio));
        System.out.println("Color: " + color);
        double area1 = 2 * PI * radio * radio;
```

Extract
- Extract interface...
- Extract to method
- Extract to local variable (replace all occurrences)
- Extract to field
- Extract to local variable
- Extract to constant

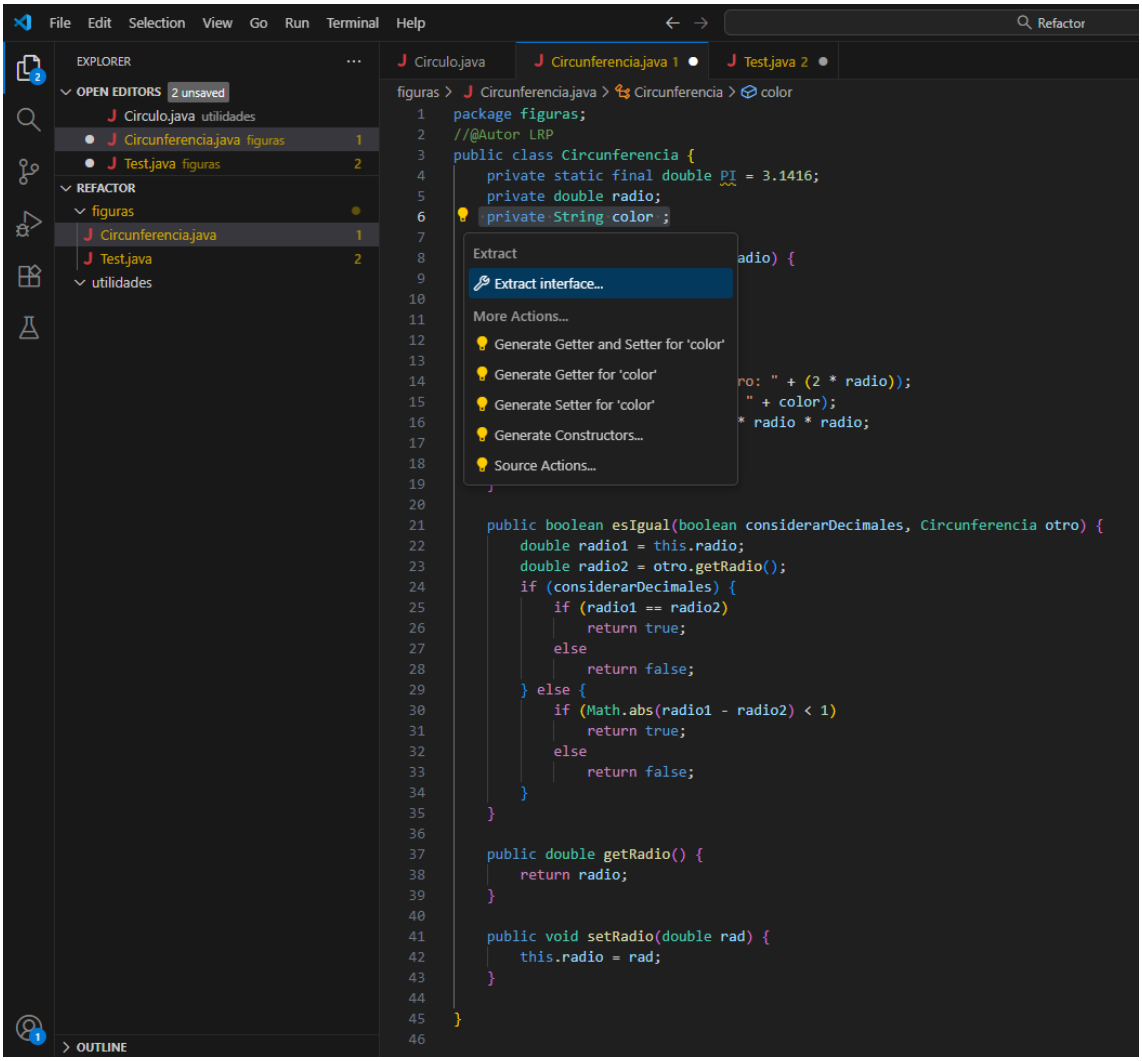
Inline
- Inline Constant

More Actions...
- Introduce Parameter...
- Add Javadoc comment
- Add final modifier for 'area1'
- Source Actions...

```java
    }

    public void setRadio(double rad) {
        this.radio = rad;
    }

}
```


File Edit Selection View Go Run Terminal Help
Refactor
EXPLORER
OPEN EDITORS 2 unsaved
Circulo.java utilidades
Circunferencia.java figuras 1
Test.java figuras 2
REFACTOR
figuras
Circunferencia.java 1
Test.java 2
utilidades
OUTLINE
Circulo.java
Circunferencia.java 1
Test.java 2
figuras > Circunferencia.java > Circunferencia > imprimir()
package figuras;
//@Autor LRP
public class Circunferencia {
private static final double PI = 3.1416;
private double radio;
private String color ;
public Circunferencia(double radio) {
this.radio = radio;
}
public void imprimir() {
color="rojo";
System.out.println("Diámetro: " + (2 * radio));
System.out.println("Color: " + color);
double area1 = 2 * 3.1416 * radio * radio;
double area = area1;
System.out.println(area);
}
public boolean esIgual(boolean considerarDecimales, Circunferencia otro) {
double radio1 = this.radio;
double radio2 = otro.getRadio();
if (considerarDecimales) {
if (radio1 == radio2)
return true;
else
return false;
} else {
if (Math.abs(radio1 - radio2) < 1)
return true;
else
return false;
}
}
public double getRadio() {
return radio;
}
public void setRadio(double rad) {
this.radio = rad;
}
}

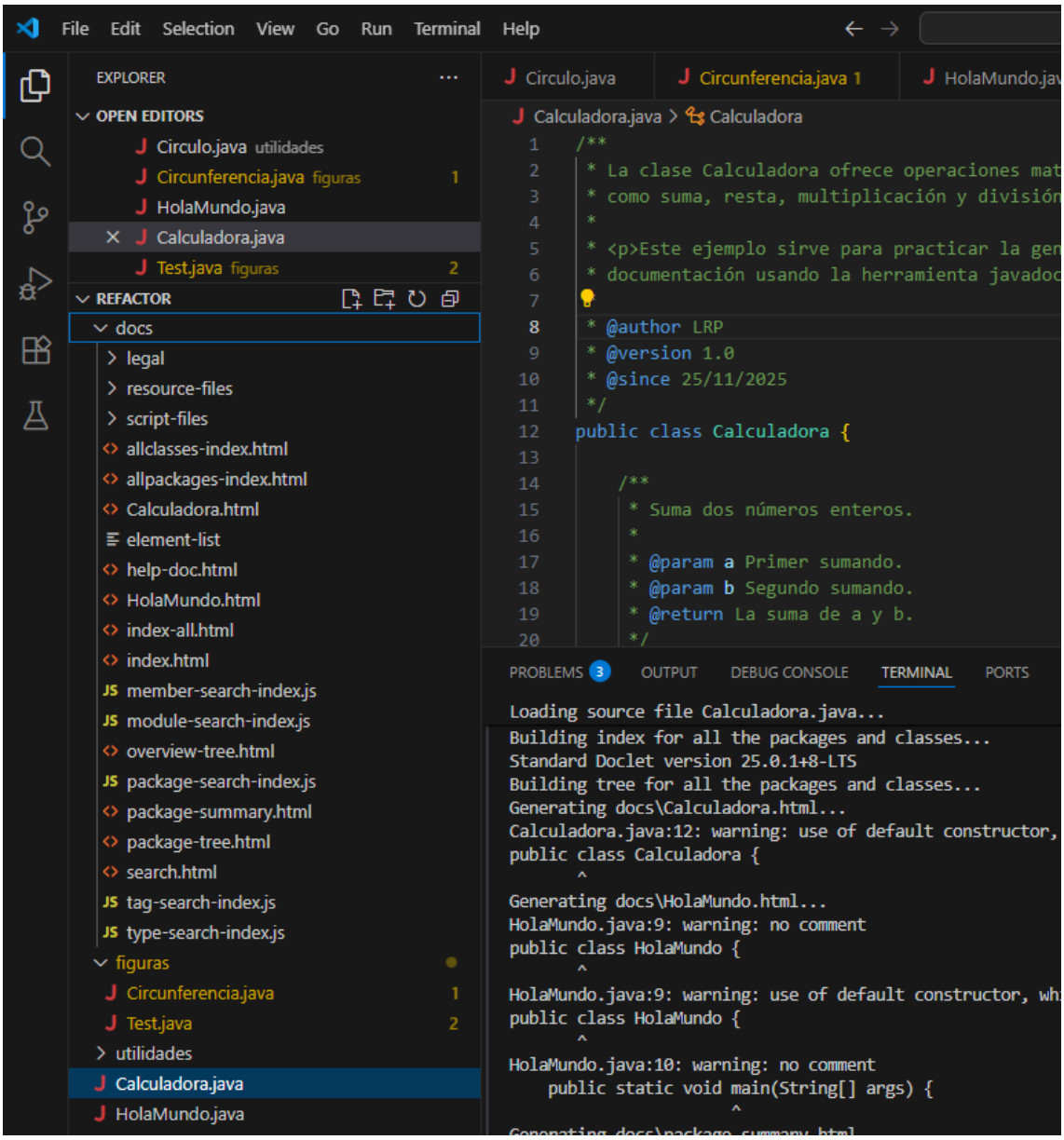
16. Intentar hacer la misma operación con el atributo "color". ¿Es posible eliminar un atributo y utilizar su valor en línea?



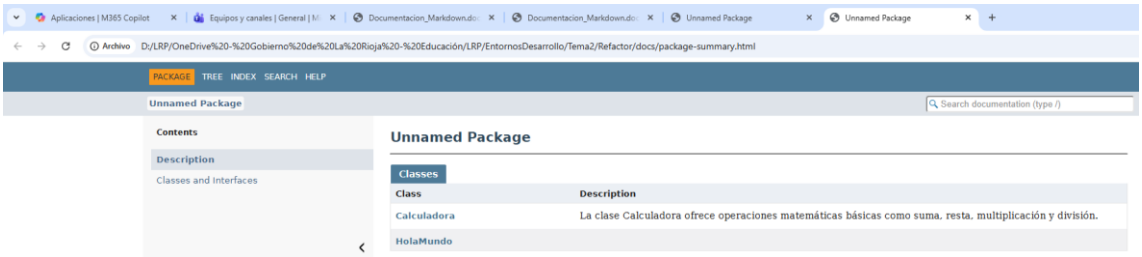
No se puede porque es un atributo y hay que hacerlo a mano.

DOCUMENTACION

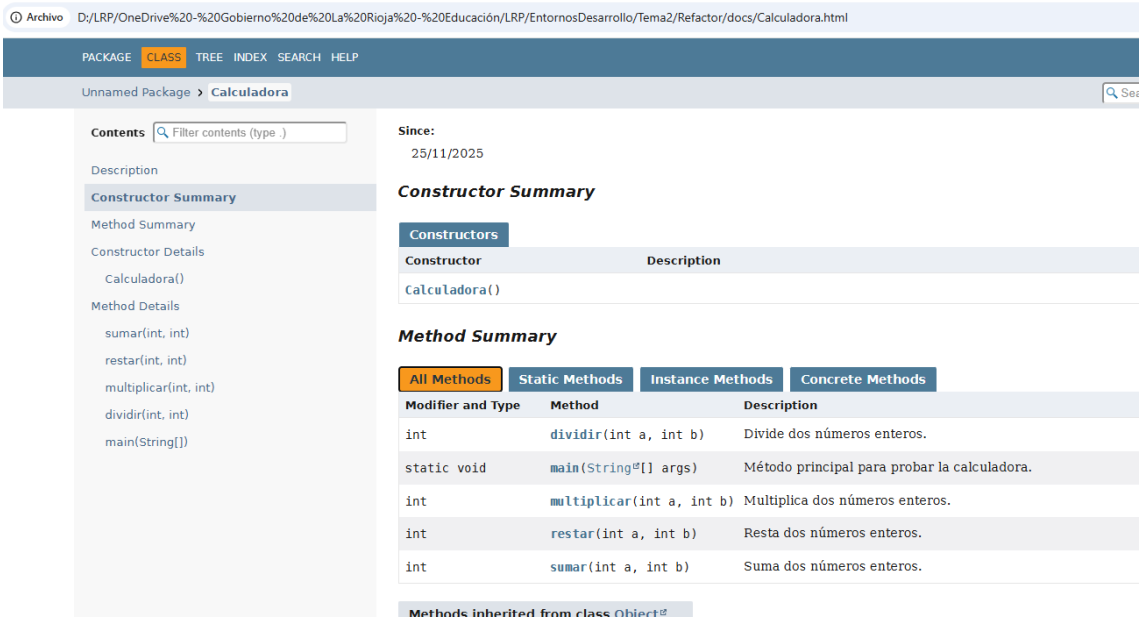
1. Captura del interior del directorio para ver lo que ha generado.



2. Captura de cómo se ve el index.html



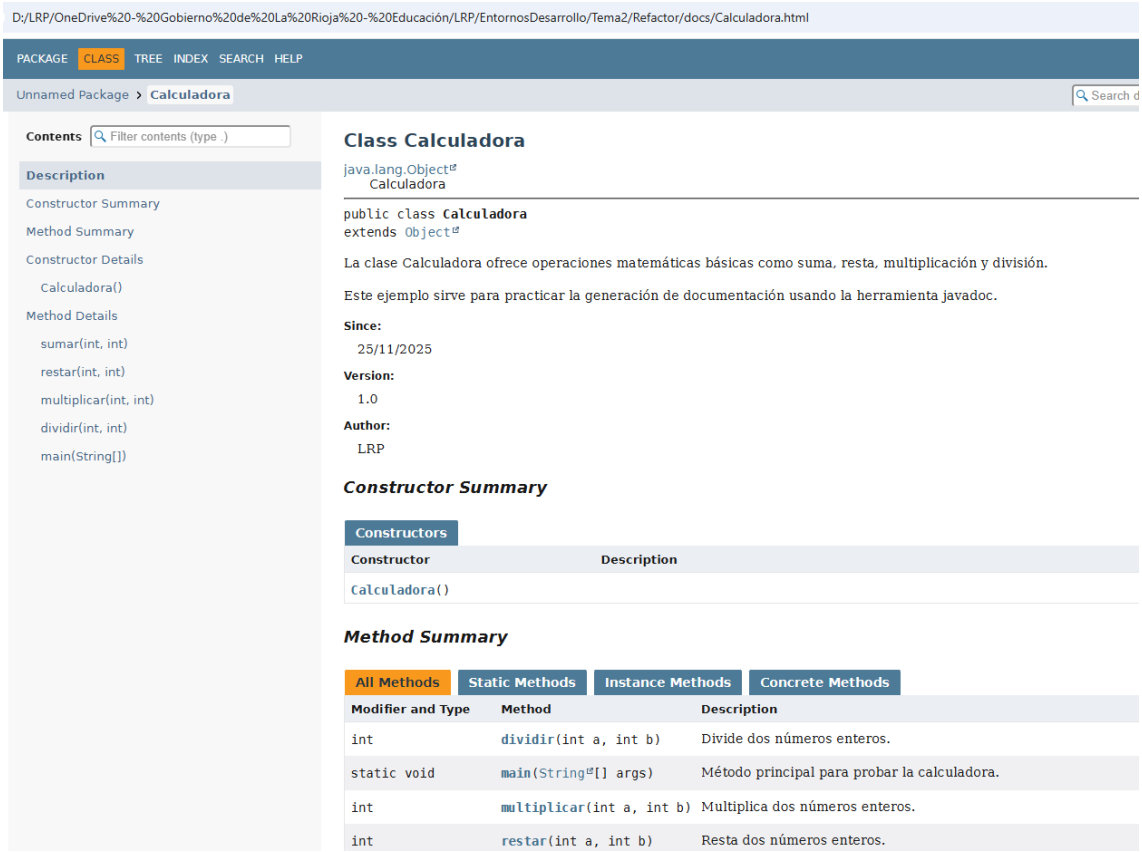
3. Captura de la documentación de la Calculadora que se vean todos los métodos.



¿Cómo lo generamos para que salga en la documentación el autor y la versión?

```
PS D:\LRP\OneDrive - Gobierno de La Rioja - Educación\LRP\EntornosDesarrollo\Tema2\Refactor> javadoc -d docs -author -version *.java
Loading source file Calculadora.java...
Loading source file HolaMundo.java...
Constructing Javadoc information...
Building index for all the packages and classes...
Standard Doclet version 25.0.1+8-LTS
Building tree for all the packages and classes...
Generating docs\Calculadora.html...
Calculadora.java:12: warning: use of default constructor, which does not provide a comment
public class Calculadora {
       ^
Generating docs\HolaMundo.html...
HolaMundo.java:9: warning: no comment
public class HolaMundo {
       ^
HolaMundo.java:9: warning: use of default constructor, which does not provide a comment
public class HolaMundo {
       ^
HolaMundo.java:10: warning: no comment
    public static void main(String[] args) {
                       ^
Generating docs\package-summary.html...
Generating docs\package-tree.html...
Generating docs\overview-tree.html...
Generating docs\allclasses-index.html...
Building index for all classes...
Generating docs\allpackages-index.html...
```

4. Captura de la nueva documentación donde se ve el autor y la versión.



D:/LRP/OneDrive%20-%20Gobierno%20de%20La%20Rioja%20-%20Educación/LRP/EntornosDesarrollo/Tema2/Refactor/docs/Calculadora.html

PACKAGE CLASS TREE INDEX SEARCH HELP

Unnamed Package > Calculadora

Contents Filter contents (type .)

- Description
- Constructor Summary
- Method Summary
- Constructor Details
  - Calculadora()
- Method Details
  - sumar(int, int)
  - restar(int, int)
  - multiplicar(int, int)
  - dividir(int, int)
  - main(String[])

Class Calculadora

java.lang.Object
  Calculadora

public class **Calculadora**
extends Object

La clase Calculadora ofrece operaciones matemáticas básicas como suma, resta, multiplicación y división.

Este ejemplo sirve para practicar la generación de documentación usando la herramienta javadoc.

Since:
25/11/2025

Version:
1.0

Author:
LRP

Constructor Summary

Constructors

Constructor	Description
Calculadora()	

Method Summary

All Methods | Static Methods | Instance Methods | Concrete Methods

Modifier and Type	Method	Description
int	dividir(int a, int b)	Divide dos números enteros.
static void	main(String[] args)	Método principal para probar la calculadora.
int	multiplicar(int a, int b)	Multiplica dos números enteros.
int	restar(int a, int b)	Resta dos números enteros.

MARKDOWN

https://www.markdowntu

Cursivas y negrita

Cursivas y negrita

Empezaremos aprendiendo dos elementos básicos a la hora de dar formato a un texto: *cursivas* y **negritas**. ¡En estas lecciones, verá partes `con formato en texto rojo`; este texto está escrito en Markdown! El Markdown no tiene un aspecto diferente al texto normal, pero de esta manera destacamos en este tutorial el texto, para que sea sencillo de ver.

Para poner una palabra o palabras en *cursiva* en Markdown, debe poner al inicio y al final de las palabra un guión bajo (`_`). Por ejemplo, `_esta_` palabra se convertiría en *cursiva*.

Para esta siguiente lección, convierta la palabra "no" en cursiva.

```
¡Escribir en Markdown _no_ es difícil!**LRP**
```

¡Escribir en Markdown *no* es difícil!**LRP**

Omitir

¡Asombroso! Buen trabajo.

De manera similar, para poner una palabra o palabras en **negrita** en Markdown, debe poner al inicio y al final de las palabra dos asteriscos (`**`). Esto `**realmente**` dará enfasis a sus puntos de vista.

En el recuadro inferior, convierta la palabra "Completaré" en negrita.

```
¡**Completaré** estas lecciones!**LRP**
```

¡Completaré estas lecciones!**LRP**

Omitir

¡Buen trabajo!

Por supuesto, puede utilizar `_tanto las cursivas como negritas_` en la misma línea. También puede abarcar `**múltiples palabras**`.

En el recuadro inferior, haga que las palabras "Por supuesto" estén en cursiva y las palabras "un poco de moxie" en negrita.

```
_"Por supuesto"_, susurró ella. Entonces, gritó: "Todo lo que
necesito es **un poco de moxie!**"**LRP**¡Escribir en
Markdown no es difícil!
```

"Por supuesto", susurró ella. Entonces, gritó: "Todo lo que necesito es **un poco de moxie!**"**LRP**¡Escribir en Markdown no es difícil!

Omitir

Encabezados

Encabezados

Echemos un vistazo a otro elemento de formato de textos: el encabezado. Los encabezados son utilizados frecuentemente en sitios web, artículos en revistas o noticias, para llamar la atención sobre una sección. Tal como su nombre indica, estos actúan como títulos o subtítulos sobre secciones.

Existen seis tipos de encabezados, en tamaños decrecientes:

Este es el encabezado uno

Este es el encabezado dos

Este es el encabezado tres

Este es el encabezado cuatro

Este es el encabezado cinco

Este es el encabezado seis

Para realizar encabezados en Markdown, debe comenzar la frase con el símbolo de la almohadilla (`#`). Debe colocar el mismo número de almohadillas como el tamaño del encabezado que quiera crear. Por ejemplo, para un encabezado de tipo uno, necesitaría utilizar una almohadilla (`# Encabezado uno`), mientras que para un encabezado tres, necesitaría utilizar tres (`### Encabezado tres`).

Para esta siguiente lección, de a cada encabezado el tamaño correcto.

```
#Encabezado uno**LRP**
##Encabezado dos
###Encabezado tres
####Encabezado cuatro
#####Encabezado cinco
######Encabezado seis
```

Encabezado uno**LRP**

Encabezado dos

Encabezado tres

Encabezado cuatro

Encabezado cinco

Encabezado seis

Omitir

¡Muy bien!

Le corresponde a usted decidir cuando es apropiado utilizar uno u otro tamaño de encabezado. En general, los encabezados uno o seis se utilizan lo menos posible.

No puede hacer que el encabezado este en negrita, pero sí se puede poner en cursiva ciertas palabras. En el recuadro inferior, convierta la primera línea en un encabezado de nivel cuatro, y ponga en cursiva el nombre del libro:

```
####El Simbolismo Colombiano en _Cien años de soledad_
**LRP**

Algunas palabras sobre el libro _Cien años..._.
```

El Simbolismo Colombiano en *Cien años de soledad* **LRP**

Algunas palabras sobre el libro *Cien años...*

Omitir

Enlaces

Enlaces

Ahora aprenderemos cómo crear enlaces a otros sitios web.

Existen dos tipos diferentes de enlaces en Markdown, pero ambos funcionan de forma similar. El primer tipo de enlace es el llamado *enlace en línea*. Para crear un enlace en línea, hay que encerrar el texto del enlace entre corchetes (`[ ]`), y después encerrar el enlace entre paréntesis (`( )`). Por ejemplo, para crear un enlace a www.github.com con un texto del enlace donde diga, ¡Visita GitHub!, debería escribir esto en Markdown: `[¡Visita GitHub!](www.github.com)`.

En el recuadro inferior, crea un enlace a www.google.com, con un texto que diga "Búscalo"."

```
[Búscalo.](www.google.com)**LRP**
```

Búscalo.**LRP**

Omitir

¡Buen trabajo!

También puede dar énfasis a los textos de los enlaces, si desea. En el recuadro inferior, ponga en negrita las palabras "realmente, realmente", y enlace toda la frase a este enlace www.dailykitten.com. Asegúrese que el texto en negrita está dentro del texto del enlace entre corchetes.

```
[Usted **realmente, realmente** quiere ver
esto.](www.dailykitten.com)**LRP**
```

Usted **realmente, realmente** quiere ver esto.**LRP**

Omitir

¡Fantástico!

Aunque podría ser algo poco común, también puede crear enlaces dentro de encabezados.

Para el siguiente tutorial, convierta el texto en un encabezado de tipo cuatro y convierta la frase "la BBC" en un enlace a www.bbc.com/news

```
####Las últimas noticias desde la
[BBC](www.bbc.com/news)**LRP**
```

Las últimas noticias desde la BBC**LRP**

Omitir

Imagenes

Imágenes

Si ya sabe cómo crear enlaces en Markdown, también puede incluir imágenes. La sintaxis es muy similar.

Las imágenes también tienen dos estilos, igual que los enlaces y ambos funcionan de manera similar. La diferencia entre los enlaces y las imágenes es que en el código de las imágenes están precedidas por un símbolo de exclamación (`!`).

El primer estilo de imagen es llamado, un *enlace en línea a una imagen*. Para crear un enlace en línea a una imagen, escriba primero un símbolo de exclamación (`!`), encierre el texto entre corchetes (`[ ]`), y después encierre el enlace entre paréntesis (`( )`). (El texto entre corchetes es una palabra o frase que sirve para describir la imagen para personas ciegas.)

Por ejemplo, para crear un enlace en línea a una imagen a https://octodex.github.com/images/bannekat.png, con un texto que diga, Benjamin Bannekat, debería escribir esto en Markdown: `![Benjamin Bannekat](https://octodex.github.com/images/bannekat.png)` .

En el recuadro inferior, convierta el enlace en una imagen y rellene el texto entre corchetes para que diga "Un bonito tigre":

```
![Un bonito tigre](https://upload.wikimedia.org/wikipedia/commons/5/56/Tiger.50.jpg)**LRP**
```

LRP

Omitir

¡Muy bien!

Aunque no es *necesario* añadir el texto entre corchetes para que funcione de igual manera, hará que el contenido creado sea accesible para su audiencia, incluyendo personas que son ciegas, utilizan lectores de pantalla, o no tienen conexiones rápidas de internet.

Para las imágenes incluidas como referencia, seguiremos el mismo patrón que para los enlaces de referencia. Deberá preceder el código de Markdown con un símbolo de exclamación, después dos corchetes para el texto y dos corchetes más para la etiqueta de la imagen. Al final de su página de Markdown, definirá la imagen correspondiente para esa etiqueta, de esta manera: `![El padre fundador][Padre]` Al final de su página de Markdown, definirá una imagen para la etiqueta, de esta manera: `[Padre]: http://octodex.github.com/images/founding-father.jpg` .

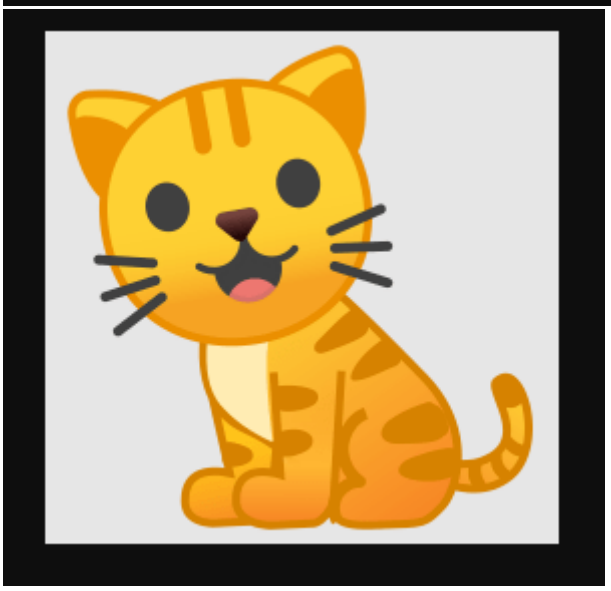
En el recuadro inferior, hemos colocado algunas imágenes en referencia; necesitará completarlas, como en la lección anterior. Llame a la etiqueta de la primera referencia "Negro", y enlácela a `https://upload.wikimedia.org/wikipedia/commons/a/a3/81_INF_DIV_SSI.jpg` ; haga que la segunda imagen apunte a `http://icons.iconarchive.com/icons/google/noto-emoji-animals-nature/256/22221-cat-icon.png` .

```
[Gato negro][Negro]

[Gato naranja][Naranja]**LRP**

[Negro]: https://upload.wikimedia.org/wikipedia/commons/a/a3/81_INF_DIV_SSI.jpg
[Naranja]:http://icons.iconarchive.com/icons/google/noto-emoji-animals-nature/256/22221-cat-icon.png
```

Gato negro

Gato naranja**LRP**

Omitir



Citas

Citas

Si necesita hacer una llamada de atención a una cita de otra fuente, o incluir una cita de una revista, entonces la sintaxis *cita* de Markdown's será útil. Una cita es una frase o párrafo que a la que se la ha dado un formato específico para llamar la atención del lector. Por ejemplo:

> "El pecado de no hacer nada es el más mortal de los siete pecados. Se ha dicho que para que los hombres malvados logren su propósito, solo es necesario que los buenos hombres no hagan nada".

Para crear una cita, todo lo que tiene que hacer es añadir al inicio de la línea el símbolo de "mayor que" (`>`). Por ejemplo:

```
> "En un abrir y cerrar de ojos se descalzó, se metió las medias en los bolsillos y se colgó al
hombro los zapatos de lona, atándolos por los cordones. Cogió un palo puntiagudo abandonado por el mar y roído por las
sales, y descendió por la rampa del rompeolas"
```

En el recuadro inferior, convierta el texto del libro en una cita:

```
Leí esta interesante cita hace unos días:**LRP**

>"Sus ojos lo habían llamado y su alma había saltado a la
llamada. ¡Vivir, errar, caer, triunfar, recrear la vida de la
vida!"
```

Leí esta interesante cita hace unos días:**LRP**

> "Sus ojos lo habían llamado y su alma había saltado a la llamada. ¡Vivir, errar, caer, triunfar, recrear la vida de la vida!"

Omitir

¡Maravilloso!

También puede colocar un símbolo en cada línea de la cita. Esto es particularmente útil si su cita ocupa múltiples párrafos. Por ejemplo:

```
> Al hablar, parecía como si sus palabras estuviesen hiriendo alguna profunda cuerda de su propia alma.
¿Había  hablado de sí mismo, de sí mismo tal como era o tal como deseaba ser? Stephen observó por algunos
instantes el rostro de su amigo. Había una fría tristeza en aquel rostro. Había hablado de sí mismo;
era el temor de su propia soledad.
>
> —¿De quién estás hablando? —preguntó por fin Stephen.
>
> Cranly no contestó.
```

Cabe destacar que incluso las líneas en blanco deben contener el símbolo. Esto asegura que toda la cita está agrupada en conjunto.

En el recuadro inferior, haga que todo sea una cita completa insertando un símbolo "mayo que" en cada línea.

```
**LRP**
>Allá en otros tiempos (y bien buenos tiempos que eran),
había una vez una vaquita (¡mu!) que iba por un caminito. Y
esta vaquita que iba por un caminito se encontró un niñín muy
guapín, al cual le llamaban el nene de la casa...

Este era el cuento que le contaba su padre. Su padre le
miraba a través de un cristal: tenía la cara peluda.

El era el nene de la casa. La vaquita venía por el caminito
donde vivía Betty Byrne: Betty Byrne vendía trenzas de azúcar
al limón.
```

LRP

> Allá en otros tiempos (y bien buenos tiempos que eran), había una vez una vaquita (¡mu!) que iba por un caminito. Y esta vaquita que iba por un caminito se encontró un niñín muy guapín, al cual le llamaban el nene de la casa...

Este era el cuento que le contaba su padre. Su padre le miraba a través de un cristal: tenía la cara peluda.

El era el nene de la casa. La vaquita venía por el caminito donde vivía Betty Byrne: Betty Byrne vendía trenzas de azúcar al limón.

Omitir

Listas

Listas

Este tutorial trata sobre cómo crear listas en Markdown.

Existen dos tipos de listas en el universo conocido: sin orden y ordenadas. Esa es una manera muy vistosa de decir que existen listas en las que delante de cada elemento hay simplemente un punto y listas precedidas de números.

Para crear una lista sin orden, deberá escribir delante de cada elemento de la lista un asterisco (`*`). Cada elemento de la lista tendrá su propia línea. Por ejemplo una lista de la compra en Markdown tendrá un aspecto similar a este:

```
* Leche
* Huevos
* Salmón
* Mantequilla
```

Esta lista en Markdown al ser renderizada tendrá el siguiente aspecto:

- Leche
- Huevos
- Salmón
- Mantequilla

En el recuadro inferior, convierta todas las palabras separadas mediante una coma en una lista.

```
**LRP**

* Harina
* Queso
* Tomates
```

?

LRP

- Harina
- Queso
- Tomates

Omitir

Párrafos

Markdown tiene varias formas de dar formato a los párrafos.

Tomemos algunas líneas de una poesía como ejemplo. Supongamos que queremos escribir texto que tenga la siguiente apariencia:

¿Qué me contradigo?
Sí, me contradigo. Y ¿qué?,
(Yo soy inmenso y contengo multitudes.)

Ahora, podría pensar que simplemente escribiendo cada verso en su propia línea sería suficiente para resolver el problema:

```
¿Qué me contradigo?
Sí, me contradigo. Y ¿qué?,
(Yo soy inmenso y contengo multitudes.)
```

¡Desafortunadamente, estaría equivocado! Este texto en Markdown una vez renderizado simplemente se mostraría todo en una única línea: ¿Qué me contradigo? Sí, me contradigo. Y ¿qué?,(Yo soy inmenso y contengo multitudes.).

Si fuerza a insertar una nueva línea, al final esto rompería el conjunto:

```
¿Qué me contradigo?

Sí, me contradigo. Y ¿qué?,

(Yo soy inmenso y contengo multitudes.)
```

Esto es lo que es conocido como *ruptura dura*; lo que nuestra poesía necesita es una *ruptura blanda*. Puede conseguir esto insertando dos espacios *después* de cada nueva línea. Esto no es posible de mostrar, ya que los espacios son invisibles, pero tendría este aspecto:

```
¿Qué me contradigo?··
Sí, me contradigo. Y ¿qué?,..
(Yo soy inmenso y contengo multitudes.)
```

Cada punto (`·`) representa un espacio del teclado.

Probemos esta técnica. En el recuadro inferior, inserte el número necesario de espacios para hacer que el poema se renderice de manera correcta:

```
**LRP**
Aquí estoy
en el lagar de la sidra,
probando la pulpa melosa y pardusca
y chupando con una paja el jugo fermentado.
```

?

LRP
Aquí estoy
en el lagar de la sidra,
probando la pulpa melosa y pardusca
y chupando con una paja el jugo fermentado.

¡Gran trabajo!

Además de dar formato a las poesías, uno de los usos más comunes de estas rupturas débiles es a la hora de dar formato en listas de párrafos. Recordemos en la lección anterior que insertamos una nueva línea para múltiples párrafos dentro de una lista.

En el recuadro inferior, en vez de utilizar rupturas duras, separe los sub párrafos con rupturas débiles:

```
**LRP**
1. Cascar tres huevos en un recipiente.
Ahora, va a cascar los huevos de forma que no sea un
desastre.
¡Si _causa_ un desastre, utilice papel de cocina o un paño
para limpiarlo!

2. Agregar un litro de leche al recipiente.
 Básicamente, tome la misma recomendación que en el punto
anterior: no ensucie todo, pero si lo hace ¡límpielo!
```

LRP
1. Cascar tres huevos en un recipiente.
Ahora, va a cascar los huevos de forma que no sea un desastre.
¡Si *causa* un desastre, utilice papel de cocina o un paño para limpiarlo!

1. Agregar un litro de leche al recipiente.
   Básicamente, tome la misma recomendación que en el punto anterior: no ensucie todo, pero si lo hace ¡límpielo!

Omitir

¡Felicidades!

¡Ha completado todas las lecciones!

Lo crea o no, sólo *acabamos de empezar* a explorar lo que se puede lograr con Markdown. Hay muchas implementaciones "extendidas" de Markdown que dan soporte a formatos como tablas, listas de definición, notas a pie de página y más. Ya que no son estándares, no son esenciales para aprender los conceptos básicos, que hemos presentado aquí.

Si le gustaría aprender más sobre esas implementciones de Markdown, está invitado/a a explorar cualquiera de estas otras apicaciones y tutoriales sobre Markdown. Aquí hay algunas opciones:

- https://daringfireball.net/projects/markdown/
- https://spec.commonmark.org/dingus/
- https://johnmacfarlane.net/babelmark2/faq.html
- https://www.markdownguide.org
- https://dave.autonoma.ca/blog/2019/05/22/typesetting-markdown-part-1/
- http://idratherbewriting.com/2013/06/04/exploring-markdown-in-collaborative-authoring-to-publishing-workflows/
- https://en.wikipedia.org/wiki/Markdown#Example
- https://docs.gitlab.com/ee/user/markdown.html
- https://docs.github.com/en/github/writing-on-github/basic-writing-and-formatting-syntax